# XFOIL$_{\mathbf{AD}}$

Adjoint-based optimization of hydrofoil sections
XFOIL analysis and exact adjoint gradients

User guide

Matthieu Sacher*
Arts et Métiers Institute of Technology, École Navale, IRENav, France

October 2026

## Contents

**1 Overview** **2**
1.1 Package contents . . . 3

**2 Installation and first steps** **3**
2.1 Python environment . . . 3
2.2 Checking the installation . . . 3
2.3 Serial or parallel computation . . . 3

**3 Getting started: analyzing a section** **4**
3.1 A `Problem` is also used for analysis . . . 4
3.2 Low-level access: `XFoilAD` . . . 4
3.3 Sensitivities with respect to the nodes and the variables . . . 6
3.4 Polars with a flap . . . 8

**4 Formulation of the optimization problem** **9**
4.1 Design variables . . . 9
4.2 Objective . . . 10
4.3 CL mode and $\alpha$ mode . . . 10
4.4 Constraints . . . 10
4.5 Scaling and bounds . . . 10
4.6 SLSQP . . . 11
4.7 IPOPT (option) . . . 11

**5 Gradients** **11**
5.1 What is differentiated . . . 11
5.2 Accuracy and settings . . . 11
5.3 Checking the gradients . . . 12

**6 Parametrizations** **13**
6.1 B-splines (`BSplineAirfoil`) . . . 13
6.2 CST (`CSTAirfoil`) . . . 13

*Parts of the implementation and of the documentation were developed with AI coding assistants.

**7 Flow conditions and transition** **14**
7.1 Parameters of an operating point . . . 14
7.2 Ncrit and turbulence Tu . . . 14
7.3 Dimensional chord, speed and Reynolds number . . . 15

**8 Constraints** **16**
8.1 Hydrodynamic constraints . . . 16
8.2 Geometric constraints . . . 16

**9 Trailing-edge flap and hinge moment** **17**
9.1 Declaring a flap . . . 17
9.2 Deflection geometry . . . 17
9.3 Hinge position as a design variable . . . 18
9.4 Hinge moment . . . 18
9.5 Example: shape and deflections optimized together . . . 19
9.6 Example: limited hinge moment . . . 20

**10 Shape optimization examples** **21**
10.1 One point, cavitation constraint . . . 21
10.2 Three points, structural constraints . . . 22
10.3 Many variables . . . 24
10.4 Symmetric section and skin-friction constraint . . . 25

**11 Robust optimization** **28**
11.1 Uncertain incidence at imposed $C_L$ . . . 29
11.2 Double uncertainty: incidence and turbulence . . . 31
11.3 Samples per condition and CVaR per group . . . 32

**12 Experimental validation** **33**

**13 Python API reference** **34**

**14 Command line and JSON file** **35**

**15 Computational cost, cache and parallelism** **37**

**16 Robustness, limitations and troubleshooting** **37**

**17 Related tools and licence** **39**

# 1 Overview

$\text{XFOIL}_{\text{AD}}$ optimizes the shape of a hydrofoil section (foil, rudder, fin, propeller or tidal-turbine blade) with a gradient-based algorithm. The hydrodynamic analysis is that of XFOIL (M. Drela): a panel method coupled with an integral boundary layer, with $e^N$ transition. The gradients are computed with an exact **adjoint method**, boundary layer included. The cost of a gradient is therefore *independent of the number of variables*: about 3 to 4 times that of an analysis, whether there are 10 or 150 variables.

**What you can do.**

- Minimize $\sum_i w_i \, f_i / f_i^0$ over one or several operating points, with $f \in \{$`cd`, `cdp`, `cdf`, `cd_cl`, `ld`, `cm`, `cmh`, $\dots\}$.
- Impose at each point either the **CL** ($\alpha$ is then an output) or $\alpha$.
- Set per point Re, Mach, **Ncrit or Tu %**, the forced transition `xtr`.
- **Optimize the deflection of a trailing-edge flap** together with the shape, one deflection per point or one shared between points; at imposed CL, XFOIL solves for $\alpha$ while the optimizer chooses the deflection.
- Constrain, per point, `cpmin` (cavitation inception), **minimum skin friction** `cfmin`, `alpha`, `cl`, `cm`, `cd`, the **hinge moment** `cmh`, etc.; and the geometry: thickness, camber, spar box, area, moments of inertia, section modulus, leading-edge radius, trailing-edge angle, smoothness.
- Declare points that carry **only constraints** (`in_objective=False`): for instance $C_f \geq 0$ at a manoeuvre $C_L$ and $T_u$, without minimizing the drag there.
- Impose a **symmetric section** (`symmetric=True`: rudder, fin, keel).
- Export the adjoint **sensitivities** $\partial C_L/\partial x_j$, $\partial C_L/\partial y_j$, the same for $C_D$, and the matrices $\partial C_{p,i}/\partial x_j$, $\partial C_{p,i}/\partial y_j$ (and $C_f$), with respect to the nodes and to the design variables.
- Run a **robust** optimization with respect to an uncertain parameter (Tu, incidence around the trim, or both), on the mean or on the CVaR, with one CVaR per condition (`incidence_samples`).
- **Size** the section: chord derived from an imposed moment of inertia or section modulus, Reynolds number following the chord (speed in m/s or in knots), objective in dimensional drag $c\,C_D$ (Section 7.3).
- Get the exact derivatives with respect to **Re and Ncrit**, and the sensitivities of the $C_p$, $C_f$ distributions with respect to the design variables in **tangent mode** (one pass per variable).
- Rely on an **experimental validation** (NACA 0012, Ladson and Abbott, Section 12).
- Choose the parametrization (B-spline or CST) and the optimizer (SLSQP or IPOPT).

**Organization of the guide.** Sections 2 and 3 show the installation and the analysis of a section. Sections 4 to 8 describe the formulation of the problem and the gradients. Sections 9 to 11 give complete optimization examples with their results. The end of the guide contains the API reference, the command line, the computational cost and troubleshooting. Each example is a complete Python script, reproduced as is with its output.

### 1.1 Package contents

XFOIL$_{AD}$ is a Python package (`xfoilad/`) shipped with two libraries compiled for 64-bit Windows: `xfoilad.dll` (analysis and adjoint gradients) and `xfoilad_tgt.dll` (tangent mode). Everything else is in Python: parametrization, flap, constraints, optimizer, figures.

```
xfoilad/
  solver.py              analysis and gradients on the nodes: XFoilAD, XFoilTangent,
    BLState
  geometry.py            BSplineAirfoil, CSTAirfoil, FlapDeflection, NACA, .dat files
  geomfuncs.py           geometric functions (thickness, inertia, ...) and their
    gradients
  problem.py             OperatingPoint, Problem, constraints, sensitivities
  robust.py              CVaRProblem, sampling (Tu, incidence, Ncrit)
  sizing.py              dimensional chord: FixedChord, ChordFromInertia,
    ChordFromModulus
  optimize.py            SLSQP optimization (scaling, failures, history)
  ipopt.py               IPOPT optimization (optional)
  plotting.py            figure: section, convergence, Cp
  __main__.py            command line: python -m xfoilad config.json
  lib/                   compiled libraries, from the GitHub release (lib/ipopt/: IPOPT,
     to
                         download, optional)
```

## 2 Installation and first steps

### 2.1 Python environment

XFOIL$_{AD}$ needs Windows 10 or 11 (64-bit) and Python 3.10 or newer, with numpy, scipy and matplotlib (for the figures). The simplest setup is a virtual environment specific to the project:

```
python -m venv venv
venv\Scripts\python.exe -m pip install numpy scipy matplotlib
```

The compiled libraries are not in the Git repository: download `xfoilad-win64.zip` from the release page (`https://github.com/msacher90/XFOILAD/releases`) and extract `xfoilad.dll` and `xfoilad_tgt.dll` into `xfoilad/lib/`. The `xfoilad/` folder is placed next to the scripts, or its parent folder is added to the Python path:

```
import sys
sys.path.insert(0, r"C:\path\to\XFOILAD")
import xfoilad as xa
```

### 2.2 Checking the installation

```
import xfoilad as xa
x, y = xa.naca4("2412", 80)
xf = xa.XFoilAD()
xf.set_flow(re=1e6, ncrit=9.0)
r = xf.analyze(x, y, cl=0.5)
print(r.converged, r.alpha, r.cd)        # True, about 2.4 deg, 0.0060
```

### 2.3 Serial or parallel computation

By default, each (point, function) pair is computed in a separate process (half of the logical cores). On a shared workstation or a laptop, switch to serial computation:

```
set XFOILAD_WORKERS=1                          :: for the whole session (cmd)
```

or use `Problem(..., n_workers=1)` in the script. The results are identical in both cases.

# 3 Getting started: analyzing a section

## 3.1 A `Problem` is also used for analysis

A `Problem` gathers a parametrization and operating points. Without optimizing, `report` and `evaluate` give the XFOIL results at the initial design `pb.dv0`:

guide/cases/c01_analysis.py

```python
from _guide import save
import xfoilad as xa

# B-spline section fitted to a NACA 2412 (12 control points per side)
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=12)

# Three operating points: imposed CL, imposed alpha, deflected flap
points = [xa.OperatingPoint(cl=0.5, re=1e6, name="CL 0.5"),
          xa.OperatingPoint(alpha=4.0, re=1e6, tu=0.5, name="alpha 4"),
          xa.OperatingPoint(cl=0.5, re=1e6, flap=5.0, name="flap 5 deg")]
pb = xa.Problem(param, points)

print(pb.report(pb.dv0))                       # table of results
ev = pb.evaluate(pb.dv0, gradients=False)      # direct access to the values
r = ev["results"][0]
print(f"alpha = {r.alpha:.3f} deg, CD = {r.cd:.6f}, upper-side transition x/c = {r.
    xtr_top:.3f}")

save("c01", report=pb.report(pb.dv0), alpha=r.alpha, cd=r.cd, xtr=r.xtr_top)
```

output

```
       CL 0.5: Re=1e+06  M=0  Ncrit=9.00 (Tu=0.0701%)
      alpha 4: Re=1e+06  M=0  Ncrit=4.29 (Tu=0.5%)
    flap 5 deg: Re=1e+06  M=0  Ncrit=9.00 (Tu=0.0701%)  flap=5 deg
         point   alpha      CL        CD       CDf       CM     L/D   Cpmin     Cfmin xtr_up xtr_lo    flap
        Cmh
       CL 0.5   2.470  0.5000  0.006010  0.004210  -0.0490    83.2  -0.884 -1.44e-05  0.501  0.980   0.000
    0.00325
      alpha 4   4.000  0.6559  0.007735  0.005315  -0.0466    84.8  -1.261  3.17e-04  0.273  0.957   0.000
    0.00350
    flap 5 deg  -0.781  0.5000  0.005447  0.003973  -0.1115    91.8  -0.686  2.12e-05  0.802  0.626   5.000
    0.00717
  flap: hinge x/c=0.75, y/t=0.5 (y=0.0128), blend=0.03
  geometry: t/c max=0.1213  area=0.08225  Ixx=6.993e-05  Iyy=4.543e-03
alpha = 2.470 deg, CD = 0.006010, upper-side transition x/c = 0.501
```

For each point, the table gives the incidence, the coefficients, $C_{p,\min}$, the transition location on each side, the flap deflection and the hinge moment $C_{mh}$ (Section 9). The third point shows the effect of a flap deflected by 5° at fixed CL: $\alpha$ decreases by 3.25°, the upper-side transition moves back from 0.50 to 0.80, $C_D$ decreases by 9 % and $C_M$ goes from $-0.049$ to $-0.112$.

## 3.2 Low-level access: `XFoilAD`

`XFoilAD` works directly on the nodes, without parametrization. It provides the analysis (`analyze`) and the adjoint gradient of a combination of the outputs with respect to all the nodes (`adjoint`):

guide/cases/c02_low_level.py

```
from _guide import save
import time
import numpy as np
import xfoilad as xa

x, y = xa.naca4("0012", 80)                    # 159 nodes, counter-clockwise
xf = xa.XFoilAD()                              # one instance per process
xf.set_flow(re=1e6, ncrit=9.0)
xf.set_tolerance(1e-11)

t0 = time.perf_counter()
r = xf.analyze(x, y, cl=0.5)                   # primal, CL mode
t_primal = time.perf_counter() - t0
t0 = time.perf_counter()
gx, gy, gcl = xf.adjoint(x, y, cl=0.5, seed={"cd": 1.0})   # dCD/dx, dCD/dy, dCD/dCL
t_adj = time.perf_counter() - t0

# check on one upper-side node with central finite differences
i, h = 40, 1e-6
e = np.zeros_like(y)
e[i] = h
fd = (xf.analyze(x, y + e, cl=0.5).cd - xf.analyze(x, y - e, cl=0.5).cd) / (2 * h)
print(f"CD = {r.cd:.6f}  alpha = {r.alpha:.3f}  converged = {r.converged}")
print(f"dCD/dy[{i}] adjoint = {gy[i]:.6e}   FD = {fd:.6e}")
print(f"dCD/dCL = {gcl:.5f}   primal {t_primal:.3f} s, adjoint {t_adj:.3f} s")

save("c02", cd=r.cd, alpha=r.alpha, node=i, adj=gy[i], fd=fd, dcd_dcl=gcl,
     t_primal=t_primal, t_adj=t_adj, n=len(x))
```

output

```
CD = 0.007959  alpha = 4.595  converged = True
dCD/dy[40] adjoint = 2.221312e-04   FD = 2.220890e-04
dCD/dCL = 0.00902   primal 0.025 s, adjoint 0.052 s
```

One adjoint pass (0.052 s) costs two to three analyses (0.025 s) and gives the 318 derivatives $\partial C_D/\partial x_j$, $\partial C_D/\partial y_j$ at once. The deviation from the finite difference ($2 \cdot 10^{-4}$ relative) is that of the finite difference itself in CL mode. `gcl`, the derivative with respect to the target value, is here $\partial C_D/\partial C_L$ at fixed shape.

`Result` also contains `cf`, the skin-friction coefficient $C_f = \tau_w/(\frac{1}{2}\rho V_\infty^2)$ at each node, signed in the direction of the boundary-layer flow from the stagnation point: $C_f < 0$ indicates separated flow (laminar bubble or turbulent separation). It is zero in inviscid flow. `adjoint` accepts a nodal seed `cf_seed`, like `cp_seed`.

With `adjoint(..., flow=True)`, the same pass also gives the derivatives with respect to the Reynolds number and to the two $N_{crit}$ (upper side, lower side) set by `set_flow`: `gx, gy, gop, g = xf.adjoint(..., flow=True)`, with `g["re"]` and `g["ncrit"]`. They are exact (deviation of $10^{-6}$ from finite differences), and are used when Re depends on the chord (Section 7.3) or for the sensitivity to turbulence: $\partial F/\partial T_u = \partial F/\partial N_{crit} \cdot dN_{crit}/dT_u$.

**Converged state and warm start.** A converged analysis returns its boundary-layer state in `r.state` (a `BLState` object: $\theta$, $\delta^*$, $C_\tau$ or amplification, $U_e$ and mass defect at each station, surface velocities, $\alpha$, $C_L$, transition intervals, number of nodes of the section). `analyze(..., init=state)` restarts from this state instead of a cold start, on the same section at another operating point ($\alpha$, $C_L$, Re, $N_{crit}$: change `set_flow` in between) or on a neighbouring section with the same number of nodes; a state computed with another number of nodes is rejected (`ValueError`). If the warm start fails, nothing is retried: `r.converged` is `False` and the caller decides.

```
xf.set_flow(re=3e6, ncrit=9.0)
st, polar = None, []
for cl in np.arange(-0.2, 1.21, 0.05):
    r = xf.analyze(x, y, cl=cl, init=st)
    if not r.converged:                 # fallback: cold start
        r = xf.analyze(x, y, cl=cl)
    if r.converged:
        st = r.state                    # start of the next point
        polar.append((r.alpha, r.cl, r.cd))
```

Gain measured on polars of 29 to 33 points (159 nodes): $\times 1.2$ to $\times 1.6$ at imposed $C_L$, with fewer failures; at imposed $\alpha$, $\times 1.1$ on the thick symmetric section but $\times 0.75$ on a NACA 2412 up to stall, where starting from a still-attached state fails and forces a cold start. $C_D$ is the same within $10^{-7}$. `get_state()` and `set_state(state)` give the same access; XFOIL$_{\mathrm{AD}}$ uses this state when a cold analysis fails (Section 16) and for the gradients of an already evaluated design (Section 15).

`XFoilTangent` is the same interface on the tangent library: `tangent(x, y, alpha|cl, dx, dy, dop, dflow)` gives the directional derivative of *all* the outputs ($C_L$, $C_D$, …, $C_p(\cdot)$, $C_f(\cdot)$) for a perturbation of the inputs, after an `analyze` on the same section. It agrees with the adjoint within $10^{-9}$.

### 3.3 Sensitivities with respect to the nodes and the variables

`Problem.sensitivities` returns, for each requested point, the derivatives of hydrodynamic functions with respect to the coordinates $(x_j, y_j)$ of the nodes seen by XFOIL, and with respect to the design vector. A scalar function (`cl`, `cd`, `cm`, `cfmin`, …) costs one adjoint pass; the distributions `"cp"` and `"cf"` cost one pass per node and give $n \times n$ matrices, $[i, j] = \partial C_{p,i}/\partial y_j$. `write_sensitivities` writes these results to CSV and `.npz` files:

guide/cases/c15_sensitivities.py

```
from _guide import save, fig, OUT
import os
import time
import numpy as np
import xfoilad as xa

# Adjoint sensitivities of CL, CD and of the Cp distribution with respect to the
# node coordinates, at imposed alpha, and with respect to the design variables
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)
pb = xa.Problem(param, [xa.OperatingPoint(alpha=4.0, re=1e6, name="a4")], n_workers=1)

t0 = time.perf_counter()
s = pb.sensitivities(functions=("cl", "cd", "cp"))[0]     # one adjoint pass per
    function and per node
t_sens = time.perf_counter() - t0
x, n = s["x"], len(s["x"])
print(f"{n} nodes: dCL/dy {s['dy']['cl'].shape}, dCp/dy {s['dy']['cp'].shape}, {t_sens
    :.1f} s")
print(f"dCL/dalpha = {s['dop']['cl']:.4f} /deg   dCD/dalpha = {s['dop']['cd']:.6f} /deg"
    )
print("dCD/d(design variables):", np.round(s["ddv"]["cd"], 4))

# check of one column of dCp/dy with finite differences
xf, j, h = pb.hydro.solver, 40, 1e-6
e = np.zeros(n)
e[j] = h
rp, rm = xf.analyze(x, s["y"] + e, alpha=4.0), xf.analyze(x, s["y"] - e, alpha=4.0)
fd = (rp.cp - rm.cp) / (2 * h)
```

```
err = np.abs(s["dy"]["cp"][:, j] - fd).max() / np.abs(fd).max()
print(f"column {j} of dCp/dy: max relative deviation from finite differences {err:.1e}")

paths = pb.write_sensitivities(os.path.join(OUT, "c15_sens"), functions=("cl", "cd", "cp
    "))
print("\n".join(os.path.relpath(p, OUT) for p in paths))

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
ile = int(np.argmin(x))
fig_, axs = plt.subplots(1, 3, figsize=(13, 3.6))
for f, ax in zip(("cl", "cd"), axs[:2]):
    g = s["dy"][f]
    ax.plot(x[:ile + 1], g[:ile + 1], label="upper side")
    ax.plot(x[ile:], g[ile:], label="lower side")
    ax.axhline(0, color="k", lw=0.5)
    ax.set_xlabel("x/c")
    ax.set_title(rf"$\partial C_{{{f[1].upper()}}}/\partial y_j$")
    ax.legend(fontsize=8)
v = np.abs(s["dy"]["cp"]).max() / 20
im = axs[2].imshow(s["dy"]["cp"], cmap="RdBu_r", vmin=-v, vmax=v)
axs[2].set_title(r"$\partial C_{p,i}/\partial y_j$ (node indices)")
axs[2].set_xlabel("j")
axs[2].set_ylabel("i")
fig_.colorbar(im, ax=axs[2], shrink=0.85)
fig_.tight_layout()
fig_.savefig(fig("c15_sens.pdf"))
save("c15", n=n, t_sens=t_sens, dcl_da=s["dop"]["cl"], dcd_da=s["dop"]["cd"], err=err,
    node=j,
     files=[os.path.relpath(p, OUT) for p in paths], ndv=pb.ndv)
```

output

```
159 nodes: dCL/dy (159,), dCp/dy (159, 159), 9.2 s
dCL/dalpha = 0.0971 /deg   dCD/dalpha = 0.000471 /deg
dCD/d(design variables): [ 0.0263 -0.0089  0.0505 -0.0425 -0.0105  0.0273  0.0244  0.0301 -0.0027
 -0.002   0.0011  0.0051 -0.0092 -0.0294 -0.0099]
column 40 of dCp/dy: max relative deviation from finite differences 1.1e-08
c15_sens\sens_a4.csv
c15_sens\sens_a4.npz
c15_sens\sens_dv.csv
```

The 161 adjoint passes (`cl`, `cd` and 159 for $C_p$) take 9 s, i.e. 0.06 s per pass. The column of $\partial C_p/\partial y$ checked with finite differences is exact within $1 \cdot 10^{-8}$.

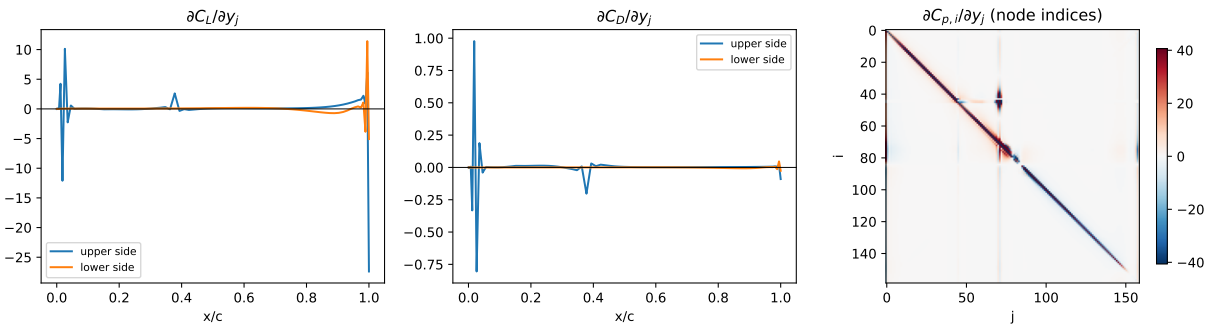



Figure 1: NACA 2412, $\alpha = 4°$, Re $= 10^6$: $\partial C_L/\partial y_j$ and $\partial C_D/\partial y_j$ along each side, and matrix $\partial C_{p,i}/\partial y_j$ (colour scale saturated at 5 % of the maximum). The diagonal reflects the local effect of a node on the pressure; the vertical bands are nodes (leading edge, vicinity of the transition) whose position acts on the whole distribution.

What you should know:

- The nodal derivatives are taken at fixed target: at fixed $\alpha$ in $\alpha$ mode, at fixed $C_L$ in CL mode ($\alpha$ then varies, and $\partial C_L/\partial y_j = 0$). `dop` gives the derivative with respect to the target ($\partial F/\partial\alpha$ per degree or $\partial F/\partial C_L$).
- With a flap, the nodes are those of the deflected section. `ddv`, the gradient with respect to the design variables, contains the whole chain: parametrization, flap, and trim angle of the points defined by `alpha_from`.
- Files written per point: `sens_<point>.csv` (one line per node: $x$, $y$, $C_p$, $C_f$, then $\partial F/\partial x$, $\partial F/\partial y$ of the scalar functions and the diagonal of the matrices) and `sens_<point>.npz` (all the arrays, full matrices and `ddv_<f>` gradients included); plus `sens_dv.csv`: one line per design variable, one column per (function, point).
- From the JSON file, a `"sensitivities"` block does the same after the optimization (Section 14).
- `method="tangent"`: the distributions `"cp"` and `"cf"` are differentiated with respect to the design variables in tangent mode, one pass per variable for all of them at once, instead of one adjoint pass per node. On a NACA 2412 with 15 variables and 3 points (including one `alpha_from` point), both methods give the same $\partial C_{p,i}/\partial d_k$ and $\partial C_{f,i}/\partial d_k$ within $10^{-10}$, in 3.9 s against 220 s. The matrices with respect to the nodes (`dx`, `dy`) are then not computed. Tangent mode is not available for a point with a flap.

### 3.4 Polars with a flap

For a polar, create a small `Problem` per point and share the solver; non-converged points are simply skipped:

guide/cases/c03_flap_polars.py

```
from _guide import save, fig
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import xfoilad as xa

param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=12)
xf = xa.XFoilAD()                       # shared solver (serial computation)
```

```
alphas = np.arange(-2.0, 9.0, 1.0)
polars = {}
for delta in (0.0, 4.0, 8.0):
    rows = []
    for a in alphas:
        pb = xa.Problem(param, [xa.OperatingPoint(alpha=a, re=1e6, flap=delta)],
                        flap_hinge=(0.75, 0.5), solver=xf, n_workers=1, verbose=False)
        try:
            ev = pb.evaluate(pb.dv0, gradients=False)
        except xa.EvaluationError:
            continue                    # point not converged: skipped
        r = ev["results"][0]
        rows.append((a, r.cl, r.cd, r.cm, pb.hinge_moments(ev)[0]))
    polars[delta] = np.array(rows)

fig_, ax = plt.subplots(1, 3, figsize=(12, 3.6))
for delta, P in polars.items():
    lab = f"flap {delta:g}°"
    ax[0].plot(P[:, 0], P[:, 1], "o-", ms=3, label=lab)
    ax[1].plot(P[:, 2], P[:, 1], "o-", ms=3, label=lab)
    ax[2].plot(P[:, 0], P[:, 4], "o-", ms=3, label=lab)
ax[0].set(xlabel="α (°)", ylabel="$C_L$")
ax[1].set(xlabel="$C_D$", ylabel="$C_L$")
ax[2].set(xlabel="α (°)", ylabel="$C_{mh}$ (hinge, / $q c^2$)")
for a in ax:
    a.grid(alpha=0.3)
ax[0].legend()
fig_.tight_layout()
fig_.savefig(fig("flap_polars.pdf"))

save("c03", polars={str(k): v for k, v in polars.items()})
```

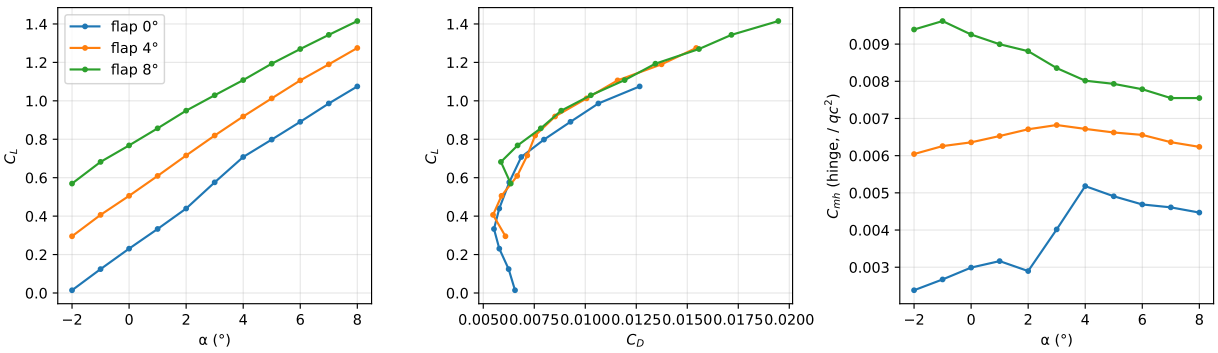



Figure 2: NACA 2412 (B-spline, 12 control points), Re = $10^6$, 25 %-chord flap deflected by 0, 4 and 8°. Left: $C_L(\alpha)$; middle: the polar $C_L(C_D)$; right: the hinge moment.

The flap shifts $C_L(\alpha)$ by 0.065 to 0.07 per degree, as predicted by thin-airfoil theory for a 25 % flap ($\approx 0.066/°$). The hinge moment increases with the deflection: the load on the flap increases and tends to raise it (sign convention, Section 9.4).

# 4 Formulation of the optimization problem

## 4.1 Design variables

The design vector $\mathbf{d} \in \mathbb{R}^n$ gathers

- the $n_s$ shape variables of the parametrization (Section 6);
- one flap deflection $\delta_k$ (in degrees) per flap variable declared by the operating points (`flap_var`, Section 9).

`pb.dv0`, `pb.ndv` and `pb.names` describe this vector; `pb.shape_dv(dv)` extracts the shape from it and `pb.flap_angles(dv)` gives the deflection of each point. Without a flap, $\mathbf{d}$ reduces to the shape variables (`pb.dv0` = `param.dv0`).

### 4.2 Objective

For $P$ points of weights $w_i$, the objective is normalized by the initial values:

$$J(\mathbf{d}) = \frac{1}{\sum_i w_i} \sum_{i=1}^{P} w_i \, \frac{f_i(\mathbf{d})}{|f_i(\mathbf{d}_0)|}, \tag{1}$$

hence $J(\mathbf{d}_0) = 1$ when all the $f_i$ are positive. The sum covers only the points of the objective: a point declared with `in_objective=False` is computed at each design but only serves its constraints (example in Section 10.4). $f$ is chosen with `objective`: `cd`, `cdf`, `cdp`= $C_D - C_{Df}$, `cm`, `alpha`, `cd_cl`= $C_D/C_L$, `ld`= $C_L/C_D$ (to be maximized: prefer `cd_cl`), `cpmin`, `cmh`, or `drag`= $c\,C_D$ (drag per unit span and per unit dynamic pressure, in m; $C_D$ without a chord model, Section 7.3).

With `Problem(..., normalize="global")`, the objective is the weighted sum normalized globally,

$$J(\mathbf{d}) = \frac{\sum_i w_i \, f_i(\mathbf{d})}{|\sum_i w_i \, f_i(\mathbf{d}_0)|}, \tag{2}$$

and not the mean of the $f_i/f_i(\mathbf{d}_0)$: this is the right form for a dimensional drag, or for a mean over samples of an uncertain parameter (the weights are then probabilities). `evaluate` also returns the values $f_i$ (`fk`) and their gradients (`fk_grad`), used by `CVaRProblem`.

### 4.3 CL mode and $\alpha$ mode

- **Imposed CL** (`OperatingPoint(cl=...)`): XFOIL's CL mode, where $\alpha$ is an unknown of the Newton method. $C_D$ is evaluated exactly at the requested $C_L$, which is generally the relevant formulation for a foil or a blade. $\alpha$ becomes an output, which can be constrained. With a free flap, there are two unknowns for one equation: XFOIL solves for $\alpha$, the optimizer chooses $\delta$.
- **Imposed $\alpha$** (`OperatingPoint(alpha=...)`), in degrees.

### 4.4 Constraints

Each constraint reads $\ell \le g(\mathbf{d}) \le u$, or $g = e$ with `equals`. It is scaled by $s = \max(|\text{bound}|, \varepsilon)$, or by `scale`. For SLSQP, each bound becomes an inequality $(g - \ell)/s \ge 0$ or $(u - g)/s \ge 0$, and an equality becomes `type="eq"`. The list of constraints is given in Section 8.

### 4.5 Scaling and bounds

`optimize()` works on the reduced variables

$$z_k = (d_k - d_{0,k})/s_k, \qquad s_k = \begin{cases} \texttt{dv_scale}\ (0.01) & \text{shape,} \\ \texttt{flap_scale}\ (1^\circ) & \text{deflection.} \end{cases} \tag{3}$$

The shape variables are bounded by $\mathbf{d}_0 \pm$ `dv_bounds`, the deflections by `flap_bounds` ($-10^\circ$ to $15^\circ$ by default). The lower bound of the leading-edge parameter $a$ is raised to $0.25\,a_0$ to keep a positive radius.

### 4.6 SLSQP

The evaluations are cached per design vector: objective, constraints and gradients share a single XFOIL computation and a single set of adjoint passes. If XFOIL does not converge on a trial shape, the returned objective is $10^3$ and the constraints $-1$: the line search backs off. `optimize` returns the *best feasible iterate* of the history (maximum violation $\leq$ `feas_tol`), not the last one.

### 4.7 IPOPT (option)

`optimize(pb, method="ipopt")` replaces SLSQP with IPOPT (interior point with filter, L-BFGS Hessian), with the same scaling and the same history. `ipopt.py` calls the IPOPT C API through ctypes: no compiled extension is needed. The library is searched for in this order: the `dll` argument, the `XFOILAD_IPOPT_DLL` variable, a copy in `xfoilad/lib/ipopt/`, then the system path. IPOPT is not included: download `Ipopt-3.14.20-win64-msvs2022-md.zip` from the coin-or Ipopt releases on GitHub (EPL-2.0) and copy the DLLs of its `bin/` folder into `xfoilad/lib/ipopt/`. Options are passed with `options={"mu_strategy": "monotone"}`.

```
res = xa.optimize(pb, method="ipopt", maxiter=80, options={"mu_strategy": "monotone"})
print(res.nfev, res.nfail)          # evaluations, rejected steps (XFOIL failures)
```

On the cases tried, IPOPT sometimes finds a slightly better optimum, but with 2 to 3 times more evaluations, and it fails when XFOIL often diverges: its barrier and restoration steps move away from the current design, into regions where XFOIL no longer converges. SLSQP remains the default choice.

## 5 Gradients

### 5.1 What is differentiated

For each operating point, the library differentiates the outputs $C_L$, $C_D$, $C_{Df}$, $C_M$, $\alpha$ and the $C_p$ and $C_f$ distributions with respect to the coordinates $(x_j, y_j)$ of all the nodes, to the target ($\alpha$ or $C_L$), to the Reynolds number and to the two $N_{crit}$. The nodes come directly from the parametrization, without repaneling. One adjoint pass gives the gradient of a combination of the outputs with respect to all the nodes at once; $\text{XFOIL}_{\text{AD}}$ then maps it exactly to the design variables (parametrization, flap, trim angle, dimensional chord).

### 5.2 Accuracy and settings

The `kiter` parameter (4 by default) sets the accuracy of the adjoint pass:

| `kiter` | max relative error ($dC_D/dy_j$, free transition) | time of one adjoint pass |
|---|---|---|
| 1 | $4 \cdot 10^{-1}$ | 0.028 s |
| 2 | $7 \cdot 10^{-3}$ | 0.033 s |
| 3 | $4 \cdot 10^{-5}$ (FD level) | 0.039 s |
| 4 | $\leq 4 \cdot 10^{-8}$ | 0.04 s |

NACA 0012, $\text{Re} = 10^6$, 159 nodes.

The skin friction $C_f$ converges more slowly: the functions that depend on it (`cfmin`, sensitivities of `cf`) use $\max($`kiter`$,$ `cf_kiter`$)$, with `cf_kiter` $= 6$ by default (error of the order of $10^{-10}$). Both are set in `Problem`.

**Seeds.** `XFoilAD.adjoint` takes a seed `seed` on $(C_L, C_D, C_{Df}, C_M, \alpha)$ and two nodal seeds, `cp_seed` and `cf_seed`; it returns the gradient of $\sum_k s_k \,\text{out}_k + \sum_j \bar{c}_j \, C_{p,j} + \sum_j \bar{f}_j \, C_{f,j}$. $C_{Dp}$, $C_D/C_L$, the aggregated $C_{p,\min}$ and $C_{f,\min}$ and the hinge moment therefore cost only one pass each. One pass

is needed per requested (point, function) pair. The gradient with respect to the flap deflections requires no additional pass.

### 5.3 Checking the gradients

guide/cases/c04_gradients.py

```
from _guide import save
import time
import numpy as np
import xfoilad as xa

param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=12)          # 19 shape variables
pts = [xa.OperatingPoint(cl=0.4, re=1e6, flap=2.0, flap_var=True, name="cruise"),
       xa.OperatingPoint(cl=0.9, re=1e6, name="high CL")]
pb = xa.Problem(param, pts, objective="cd")                      # + 1 flap variable
pb.add_constraint("cpmin", point=1, lower=-2.0)
pb.add_constraint("cmh", point=0, upper=0.01)
pb.hydro.tol = 1e-11                                            # tight Newton for the
     FD

t0 = time.perf_counter()
ev = pb.evaluate(pb.dv0)                                        # values + adjoint
    gradients
t_adj = time.perf_counter() - t0
G = np.vstack([ev["grad"]] + [J for J in ev["cgrads"]])         # objective, cpmin,
    cmh

t0 = time.perf_counter()
FD = np.zeros_like(G)
for k in range(pb.ndv):                                         # central FD, one
    variable at a time
    h = 1e-6 if k < pb.n_shape else 1e-4                        # flap in degrees
    e = np.zeros(pb.ndv)
    e[k] = h
    fp, fm = pb.evaluate(pb.dv0 + e, False), pb.evaluate(pb.dv0 - e, False)
    FD[:, k] = (np.r_[fp["f"], np.concatenate(fp["cvals"])] - np.r_[fm["f"], np.
    concatenate(fm["cvals"])]) / (2 * h)
t_fd = time.perf_counter() - t0

err = np.abs(G - FD).max(axis=1) / np.abs(FD).max(axis=1)
for name, e_ in zip(["objective", "cpmin", "cmh"], err):
    print(f"{name:10s} max relative error = {e_:.1e}")
print(f"{pb.ndv} variables: adjoint {t_adj:.2f} s, finite differences {t_fd:.1f} s")

save("c04", err=err, t_adj=t_adj, t_fd=t_fd, ndv=pb.ndv,
     flap_col=dict(adj=G[:, -1], fd=FD[:, -1]))
```

output

```
objective  max relative error = 7.7e-07
cpmin      max relative error = 2.8e-08
cmh        max relative error = 8.2e-08
20 variables: adjoint 0.55 s, finite differences 1.3 s
```

The objective (two points), $C_{p,\min}$ and $C_{mh}$ are differentiated with respect to the 20 variables (19 shape variables and one deflection) with a relative error of at most $8 \cdot 10^{-7}$. Here the finite differences cost only 1.3 s against 0.55 s for the adjoint, because an analysis takes only a few hundredths of a second; their cost grows with the number of variables, that of the adjoint does not (Section 10.3).

# 6 Parametrizations

All parametrizations are **affine** in $\mathbf{d}_s$: the geometric Jacobian is constant and exact. The nodes are ordered counter-clockwise: upper-side TE → LE → lower-side TE.

## 6.1 B-splines (`BSplineAirfoil`)

Two clamped cubic B-splines, one per side, with `n_ctrl` control points each:

- $P_0 = (0, 0)$: leading edge, common to both curves;
- $P_1 = (0, \pm a)$: vertical tangent at the LE, $a$ shared (continuous curvature, radius $r_{LE} \propto a^2$);
- $P_2 \dots P_{n-2}$: free ordinates, fixed abscissas with a cosine distribution;
- $P_{n-1} = (1, \pm t_e/2)$: trailing edge of fixed thickness ($t_e \geq 2 \cdot 10^{-5}$, a perfectly sharp TE giving a singular derivative).

Shape vector: $\mathbf{d}_s = [a,\ y_2^u..y_{n-2}^u,\ y_2^l..y_{n-2}^l]$, i.e. $1 + 2(n_{ctrl} - 3)$ variables (15 for `n_ctrl=10`, 95 for 50, 155 for 80).

```
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)
param = xa.BSplineAirfoil.from_dat("my_airfoil.dat", n_ctrl=16)     # Selig, with or
    without a name
param = xa.BSplineAirfoil.from_coordinates(x, y, n_ctrl=12, te=0.002)
x, y = param.coordinates(param.dv0)                                  # XFOIL nodes
print(param.ndv, param.n_nodes, param.names[:3], param.le_radius(param.dv0))
```

The parameters $u$ of the nodes are fixed at construction, from the arc length of the initial airfoil, with clustering at the LE and the TE (`le_cluster`, `te_cluster`); they do not depend on $\mathbf{d}$. `from_naca`, `from_dat` and `from_coordinates` fit the control points by weighted least squares ($1/\sqrt{x}$, to favour the LE). By default there are `n_side` $= \min(\max(80, 3\,n_{ctrl}), 182)$ nodes per side; the DLL accepts at most 364 nodes in total.

**Symmetric section.** With `symmetric=True`, the lower side is the mirror of the upper side: $y_k^l = -y_k^u$, and only the upper-side variables remain, $\mathbf{d}_s = [a,\ y_2^u..y_{n-2}^u]$, i.e. $1 + (n_{ctrl} - 3)$ variables (8 for `n_ctrl=10`). The nodes are exactly symmetric, at every iteration. Fitting a cambered airfoil gives the closest symmetric section, that is, its thickness distribution.

```
param = xa.BSplineAirfoil.from_naca("0012", n_ctrl=10, symmetric=True)   # 8 variables
param = xa.CSTAirfoil.from_naca("0010", order=6, symmetric=True)        # 7 variables
```

The `camber` constraint is then identically zero, and `smoothness` applies only to the upper side. A deflected flap makes the section seen by XFOIL asymmetric; the undeflected section stays symmetric. A zero-lift point is declared at $\alpha = 0$ (`OperatingPoint(alpha=0.0)`), the exact equivalent of $C_L = 0$ and more robust than CL mode. By default, a symmetric section has no node at the leading edge (`le_node=False`): two nodes, one per side, at mid-length of the first panel, so that the stagnation point does not fall on a node at $\alpha = 0$. `le_node=True` restores the node.

## 6.2 CST (`CSTAirfoil`)

Kulfan's transformation:

$$y_{u,l}(x) = C(x) \sum_{i=0}^{n} A_i^{u,l} \binom{n}{i} x^i (1-x)^{n-i} \pm x\,\frac{t_e}{2}, \qquad C(x) = \sqrt{x}\,(1-x),$$

with a cosine node distribution and $2(n+1)$ variables (`order`$=n$), $n+1$ with `symmetric=True` ($A_i^l = -A_i^u$).

```
param = xa.CSTAirfoil.from_naca("2412", order=6)                    # 14 variables
```

`le_radius` and `smoothness` are only available with B-splines. The flap and the hinge moment work with both parametrizations.

# 7 Flow conditions and transition

## 7.1 Parameters of an operating point

| Parameter | Default | Role |
|---|---|---|
| `cl` / `alpha` | — | exactly one of the two ($\alpha$ in degrees) |
| `re` | $10^6$ | chord Reynolds number ($\leq 0$: inviscid) |
| `mach` | 0 | Mach number (Kármán–Tsien correction; 0 in hydrodynamics) |
| `ncrit` | 9 | critical amplification factor; scalar or pair (upper side, lower side) |
| `tu` | — | free-stream turbulence in %, alternative to `ncrit`; scalar or pair |
| `xtr` | (1, 1) | forced transition $x/c$ (upper side, lower side); 1 = free transition |
| `flap` | 0 | flap deflection (°, trailing edge down $> 0$); initial value if free |
| `flap_var` | `None` | `True`: free deflection specific to the point; a name: shared free deflection |
| `alpha_from` | `None` | name (or index) of a point at imposed $C_L$: the point is computed at $\alpha = \alpha_{\text{trim}}$ + `dalpha` (§11.1) |
| `dalpha` | 0 | incidence offset (°) with respect to the trim |
| `weight` | 1 | weight in the objective (or probability in robust optimization) |
| `in_objective` | `True` | `False`: the point does not count in the objective, it only carries constraints |
| `speed` | `None` | speed (m/s): $Re = V\,c/\nu$ with a chord model (§7.3) |
| `name` | auto | displayed name |
| `niter` | 150 | maximum number of Newton iterations |

## 7.2 Ncrit and turbulence Tu

Transition is predicted by the $e^N$ method: it occurs where the amplification of the Tollmien–Schlichting waves reaches $N_{crit}$. XFOIL relates $N_{crit}$ to the free-stream turbulence level $T_u$ (in %) through Mack's correlation:

$$N_{crit} = -8.43 - 2.4\,\ln\Big(\frac{T_u}{100}\Big), \qquad T_u = 100\,\exp\Big(-\frac{N_{crit} + 8.43}{2.4}\Big). \tag{4}$$

| Typical environment | $T_u$ (%) | $N_{crit}$ |
|---|---|---|
| very quiet towing tank or wind tunnel | 0.03 | 11.0 |
| standard wind tunnel (XFOIL default) | 0.07 | 9.0 |
| calm sea, water tunnel | 0.1–0.2 | 8.1–6.5 |
| rough sea, swell | 0.5 | 4.3 |
| blade in a wake, disturbed flow | 1.0–2.0 | 2.6–0.96 |

```
xa.OperatingPoint(cl=0.5, re=1e6, tu=0.1)           # Ncrit = 8.15
xa.OperatingPoint(cl=0.5, re=1e6, tu=(0.1, 0.5))    # upper side / lower side
xa.OperatingPoint(cl=0.5, re=1e6, ncrit=(9, 7))
xa.ncrit_from_tu(0.07), xa.tu_from_ncrit(9.0)       # conversions
```

Giving both `tu` and `ncrit` is an error. A warning is issued if $T_u < 0.01$ % or if $N_{crit} < 1$, since this is outside the range of the correlation.

**Example.** $T_u$ sweep of a NACA 0012 at $C_L = 0.4$:

guide/cases/c07_transition.py

```
from _guide import save
import xfoilad as xa

param = xa.BSplineAirfoil.from_naca("0012", n_ctrl=12)
xf = xa.XFoilAD()
rows = []
for tu in (0.03, 0.07, 0.2, 0.5, 1.0):
    p = xa.OperatingPoint(cl=0.4, re=1e6, tu=tu)
    pb = xa.Problem(param, [p], solver=xf, n_workers=1, verbose=False)
    r = pb.evaluate(pb.dv0, gradients=False)["results"][0]
    rows.append(dict(tu=tu, ncrit=p.ncrit, xtr_top=r.xtr_top,
                     xtr_bot=r.xtr_bot, cd=r.cd, alpha=r.alpha))
    print(f"Tu={tu:4.2f}%  Ncrit={p.ncrit:5.2f}  "
          f"xtr={r.xtr_top:.3f}/{r.xtr_bot:.3f}  CD={r.cd:.5f}")
save("c07", rows=rows)
```

| $T_u$ (%) | $N_{crit}$ | $x_{tr}$ upper side | $x_{tr}$ lower side | $C_D$ |
|---|---|---|---|---|
| 0.03 | 11.04 | 0.328 | 0.976 | 0.00686 |
| 0.07 | 9.00 | 0.284 | 0.960 | 0.00702 |
| 0.2 | 6.49 | 0.218 | 0.916 | 0.00741 |
| 0.5 | 4.29 | 0.151 | 0.825 | 0.00809 |
| 1 | 2.62 | 0.106 | 0.689 | 0.00889 |

As $T_u$ increases, the transition moves forward on both sides and the drag increases by 30 % between a very quiet flow and $T_u = 1$ %.

### 7.3 Dimensional chord, speed and Reynolds number

For a mast, a fin or a rudder, the section must often provide a given bending inertia: a thinner section must then be longer, and its Reynolds number increases. The `sizing` module relates the dimensional chord $c$ (m) to the shape variables:

| Model | Chord |
|---|---|
| `FixedChord(c)` or a number | constant |
| `ChordFromInertia(I, axis="xx")` | $c = (I/I_1(\mathbf{d}))^{1/4}$, $I_1$ inertia of the unit-chord section |
| `ChordFromModulus(W)` | $c = (W/W_1(\mathbf{d}))^{1/3}$, $W_1$ section modulus at unit chord |

A point given by its speed `speed` (m/s; `xa.KNOT` = 0.5144 m/s) then has $Re = V\,c(\mathbf{d})/\nu$ at each design:

```
pts = [xa.OperatingPoint(cl=0.2, speed=8 * xa.KNOT, tu=0.2, name="course"),
       xa.OperatingPoint(cl=0.6, speed=8 * xa.KNOT, tu=0.2, in_objective=False, name="
    manoeuvre")]
pb = xa.Problem(param, pts, objective="drag", normalize="global",
                chord=xa.ChordFromModulus(2.0e-6), nu=1.19e-6)      # sea water, ~15 °C
```

The gradient accounts for $Re(\mathbf{d})$ exactly: the adjoint derivative with respect to Re is chained through $\partial Re/\partial \mathbf{d} = (Re/c)\,\partial c/\partial \mathbf{d}$, including for the trim angle of the `alpha_from` points (deviation of $7 \cdot 10^{-5}$ from finite differences). `evaluate` returns the chord (`chord`) and the Reynolds number of each point (`re`); `report` displays the current Reynolds numbers.

# 8 Constraints

## 8.1 Hydrodynamic constraints

They are added with `point=i` (index of the point), `point="name"` or `point="all"`: `cl`, `cd`, `cdf`, `cdp`, `cm`, `alpha`, `cd_cl`, `ld`, `cpmin`, `cfmin`, `cmh`. The point may be outside the objective (`in_objective=False`).

```
pb.add_constraint("cpmin", point=1, lower=-1.2)          # cavitation: Cpmin >= -sigma
pb.add_constraint("cfmin", point="manoeuvre", lower=0.0) # Cf >= 0
pb.add_constraint("alpha", point="all", upper=8.0)
pb.add_constraint("cm", point=0, lower=-0.1)
pb.add_constraint("cl", point=2, equals=0.8)             # equality (alpha mode)
pb.add_constraint("cmh", point=1, lower=-0.003, upper=0.003)
```

**Aggregated Cpmin and cavitation.** The minimum of $C_p$ is not differentiable. The Kreisselmeier–Steinhauser function is used over the nodes of `cpmin_xrange` (by default $0 \le x \le 0.98$, to exclude the TE):

$$C_{p,\min}^{KS} = -\Big[m + \tfrac{1}{\rho}\ln\sum_j e^{\rho(-C_{p,j}-m)}\Big], \quad m = \max_j(-C_{p,j}),\ \rho = 80. \tag{5}$$

It is conservative by at most $\ln N/\rho \approx 0.06$; its softmax weights are used as the `cp_seed`. Cavitation is avoided as long as $C_{p,\min} \ge -\sigma$, with $\sigma = (p_\infty - p_v)/(\frac{1}{2}\rho V^2)$ the cavitation number: at 10 m/s and 1 m depth, $\sigma \approx 2.0$.

**Minimum skin friction (`cfmin`).** Separation is read on the skin friction: $C_f < 0$ where the flow goes back upstream. The function `cfmin` is the smoothed (KS) minimum of $C_f$ over the selected nodes:

$$C_{f,\min}^{KS} = -\Big[m + \tfrac{1}{\rho_f}\ln\sum_j e^{\rho_f(-C_{f,j}-m)}\Big], \quad m = \max_j(-C_{f,j}),\ \rho_f = 5\cdot 10^4, \tag{6}$$

and the constraint `lower=0` imposes $C_f \ge 0$ on these nodes. Since $C_f$ is a few $10^{-3}$, $\rho_f$ is much larger than for $C_p$; the value stays below the true minimum by at most $\ln N/\rho_f \approx 10^{-4}$, in practice $10^{-5}$ (only the nodes close to the minimum count). The selected nodes are those of `cfmin_xrange` (by default the whole section, $0 \le x \le 1$) on the side `cfmin_side` (`"both"`, `"upper"`, `"lower"`, split at the LE), minus the two nodes around the stagnation point, where $C_f \to 0$. These settings, and `cfmin_rho`, are arguments of `Problem`, common to all points.

Two phenomena give $C_f < 0$ in XFOIL: the *laminar separation bubble* (near the LE at high $C_L$, or before transition at low Re), which reattaches, and *turbulent trailing-edge separation*, which heralds stall. $C_f \ge 0$ over the whole section forbids both; this is very restrictive at Re $\lesssim 10^6$, where bubbles are frequent. To forbid only trailing-edge separation, exclude the bubble region, for instance with `cfmin_xrange=(0.1, 1.0)` (Section 10.4). The `Cfmin` column of `report` gives the true minimum over the same nodes.

**Hinge moment.** `cmh` is defined in Section 9.4. It exists at every point, deflected or not, as soon as the `Problem` has a hinge (`flap_hinge`, $x/c = 0.75$ by default).

## 8.2 Geometric constraints

They apply to the *undeflected* section. Their gradients are analytical for the linear functions, and obtained by *complex step* ($h = 10^{-30}$, exact to machine precision) for the polygon integrals.

| Name | Definition |
|---|---|
| `thickness(xs=...)` | $t(x) = y_u(x) - y_l(x)$ at the given abscissas (linear) |
| `camber(xs=...)` | $(y_u + y_l)/2$ (linear) |
| `max_thickness` | KS-smoothed maximum ($\rho = 2000$) of $t(x)$ on $[0.02, 0.98]$ |
| `box(x_range, y_range)` | the section contains the rectangle: $y_u(x_j) \geq y_{top}$, $y_l(x_j) \leq y_{bot}$ at 15 abscissas |
| `area` | area of the polygon (TE closed by a segment) |
| `inertia(axis)` | second moment of area about the centroid: `xx` (bending, axis parallel to the chord), `yy`, `xy`, `p` $= I_{xx} + I_{yy}$ |
| `section_modulus` | $I_{xx}/d_{\max}$, with $d_{\max}$ the smoothed maximum of $\lvert y - y_G \rvert$ |
| `le_radius` | radius of curvature at the LE ($\propto a^2$, B-spline) |
| `te_angle` | TE wedge angle over the last 2 % of the chord (rad) |
| `smoothness(side)` | second differences of the control-point displacements (B-spline); §10.3 |

```
I0 = xa.geomfuncs.inertia(param, param.dv0)[0][0]
pb.add_constraint("max_thickness", lower=0.12)
pb.add_constraint("thickness", xs=[0.9], lower=0.02)       # thickness at the hinge
pb.add_constraint("box", x_range=(0.15, 0.55), y_range=(-0.035, 0.045))
pb.add_constraint("inertia", axis="xx", lower=I0)
pb.add_constraint("te_angle", lower=np.radians(4.0))
```

# 9 Trailing-edge flap and hinge moment

## 9.1 Declaring a flap

The flap is a property of the `Problem` (hinge position) and of each point (deflection):

```
pts = [xa.OperatingPoint(cl=0.3, flap_var=True, name="cruise"),       # free deflection
       xa.OperatingPoint(cl=1.0, flap_var=True, name="climb"),        # another free
    deflection
       xa.OperatingPoint(cl=0.6, flap=2.0),                           # fixed deflection
       xa.OperatingPoint(cl=0.5, tu=0.05, flap_var="rob"),            # deflection
    shared...
       xa.OperatingPoint(cl=0.5, tu=0.50, flap_var="rob")]            # ... between
    these two points
pb = xa.Problem(param, pts, flap_hinge=(0.75, 0.5), flap_bounds=(-5, 15),
                flap_blend=0.03, flap_scale=1.0)
pb.names[-3:]                      # ['flap@cruise', 'flap@climb', 'rob']
pb.flap_angles(pb.dv0)             # [0, 0, 2, 0, 0]
x, y = pb.coordinates(pb.dv0, point=2)       # deflected section seen by point 2
```

`flap_bounds` also accepts a dictionary per variable, `{"flap@climb": (0, 20), "rob": (-3, 3)}`. A shared deflection is appropriate when several points represent the same setting, for instance the samples of a robust optimization (Section 11).

## 9.2 Deflection geometry

The hinge is at $x_h$ (`flap_hinge[0]`) and at the fraction $f_h$ (`flap_hinge[1]`) of the local thickness above the lower side, like the "y/t" of XFOIL's FLAP command:

$$y_h(\mathbf{d}_s) = y_l(x_h) + f_h \left(y_u(x_h) - y_l(x_h)\right) = c_h + \mathbf{r}_h \cdot \mathbf{d}_s, \tag{7}$$

linear in the shape: the hinge follows the section, and this dependence enters the gradient. Each node rotates about $(x_h, y_h)$ by an angle $\theta_j = \delta\, w(x_j)$, where $w$ goes from 0 to 1 over $x_h \pm b$ ($b =$

`flap_blend`, 0.03 by default):

$$w = s^3(10 - 15s + 6s^2), \quad s = \mathrm{clip}\Big(\frac{x - x_h + b}{2b}, 0, 1\Big), \tag{8}$$

$$x' = x_h + \cos\theta\,(x - x_h) + \sin\theta\,(y - y_h), \qquad y' = y_h - \sin\theta\,(x - x_h) + \cos\theta\,(y - y_h). \tag{9}$$

Downstream of $x_h + b$, the flap rotates as a block (rigid rotation); upstream of $x_h - b$, nothing moves. In between, the $C^2$ ramp forms a faired hinge: the surface stays smooth, no nodes overlap on the compressed side, and the mapping is differentiable. XFOIL's FLAP command, by contrast, cuts the overlapping points and leaves a sharp corner, which is not differentiable. If a deflection nevertheless folds the surface (non-monotonic abscissas on one side), the design counts as an XFOIL failure and the line search backs off; `flap_blend` must then be increased or the bounds reduced.

**Derivatives.** With $\theta_x = \delta\, w'(x)$,

$$\frac{\partial x'}{\partial x} = \cos\theta + (y' - y_h)\,\theta_x, \quad \frac{\partial x'}{\partial y} = \sin\theta, \quad \frac{\partial y'}{\partial x} = -\sin\theta - (x' - x_h)\,\theta_x, \quad \frac{\partial y'}{\partial y} = \cos\theta, \tag{10}$$

$$\frac{\partial x'}{\partial y_h} = -\sin\theta, \quad \frac{\partial y'}{\partial y_h} = 1 - \cos\theta, \quad \frac{\partial x'}{\partial \delta} = \tfrac{\pi}{180}\, w\,(y' - y_h), \quad \frac{\partial y'}{\partial \delta} = -\tfrac{\pi}{180}\, w\,(x' - x_h). \tag{11}$$

`FlapDeflection.pullback(gx, gy)` applies these formulas to the adjoint gradient (deviation from finite differences of about $10^{-7}$).

### 9.3 Hinge position as a design variable

With `Problem(..., hinge_var=True, hinge_bounds=(0.55, 0.85), hinge_scale=0.01)`, the hinge position $x_h$ is the last design variable (`hinge_x`, initial value `flap_hinge[0]`), shared by all the points with a flap; `pb.hinge_x(dv)` returns it. Its gradient is exact and costs no additional adjoint sweep. It sums three contributions:

- the deflection map: the rotation and the faired-hinge ramp depend on $x - x_h$, so that $\partial x'/\partial x_h = 1 - \partial x'/\partial x$ and $\partial y'/\partial x_h = -\partial y'/\partial x$ (`FlapDeflection.pullback`);
- the hinge height $y_h(x_h)$, which follows the surfaces: $\partial y_h/\partial x_h$ is the slope of $(1 - f_h)\,y_l + f_h\,y_u$ at $x_h$ (central difference of step $10^{-6}$ on the linear surface operators);
- for the hinge moment, the moment arm and the two cuts of the flap contour, whose interpolation weights depend on $x_h$.

The contour changes structure when $x_h$ crosses a panel node: the hinge moment stays continuous but its derivative is piecewise. The gradients of the objective, $C_m$, $C_{p,\min}$ and $C_{mh}$ agree with central finite differences to a relative error below $4 \cdot 10^{-7}$. In JSON: `"flap": {"hinge_var": true, "hinge_bounds": [0.55, 0.85]}`.

### 9.4 Hinge moment

The hinge moment is the moment of the pressures on the flap about the hinge, per unit span and divided by $q\,c^2$. It uses the formula of XFOIL's `HMOM` (routine `MHINGE`), on the closed contour of the flap: upper side from the TE to the cut, inner face down to the hinge, inner face down to the lower-side cut, lower side to the TE:

$$C_{mh} = \sum_{\text{segments } j} \bar{C}_{p,j}\,\left[(\bar{x}_j - x_h)\,\Delta x_j + (\bar{y}_j - y_h)\,\Delta y_j\right] = \sum_j \bar{C}_{p,j}\,\frac{|\mathbf{r}_{j+1}|^2 - |\mathbf{r}_j|^2}{2}, \tag{12}$$

where $\mathbf{r}$ is the position relative to the hinge and $\bar{C}_{p,j}$ the mean of the (viscous) $C_p$ at the ends of the segment. The inner faces carry the pressure of their cut point. A uniform pressure gives a zero moment (the contour is closed).

- **Sign**: that of XFOIL, positive counter-clockwise ($x$ downstream, $y$ upward). $C_{mh} > 0$ tends to *raise* the trailing edge, hence to reduce a positive deflection.
- **Flap boundary**: the cuts are the points of the *undeflected* section at abscissa $x_h$, interpolated on the segment that contains them. They are fixed linear combinations of the nodes: they follow the flap material when it rotates.
- **Gradient**: $\partial C_{mh}/\partial C_{p,j}$ is used as the `cp_seed` of an adjoint pass; the explicit derivatives with respect to the deflected nodes and to $y_h$ are added, then the chain rule through the rotation (deviation from finite differences of about $2 \cdot 10^{-8}$).
- **Order of magnitude**: referred to the flap chord $c_f$, $C_h = C_{mh}/(c_f/c)^2$. For $c_f/c = 0.25$, Figure 2 gives $|dC_h/d\delta| \approx 0.7$/rad, the order of magnitude of thin-airfoil theory.

```
ev = pb.evaluate(dv, gradients=False)
pb.hinge_moments(ev)           # Cmh of each point
pb.hinge_y(dv)                 # height of the hinge (follows the shape)
```

### 9.5 Example: shape and deflections optimized together

Two points at imposed CL, cruise ($C_L = 0.3$) and climb ($C_L = 1.0$), each with its own deflection. The flap avoids freezing a camber compromise between the two points. On this case SLSQP tries sections whose lower side crosses the upper side near the trailing edge; XFOIL$_{\text{AD}}$ rejects them as XFOIL failures (section 16) and the line search backs off.

guide/cases/c10_flap.py

```
from _guide import save, fig, summary
import xfoilad as xa
from xfoilad.plotting import plot_result

# Shape and flap deflections optimized together; at imposed CL, XFOIL solves for alpha
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=12)
points = [xa.OperatingPoint(cl=0.3, re=1e6, flap=0.0, flap_var=True, name="cruise"),
          xa.OperatingPoint(cl=1.0, re=1e6, flap=0.0, flap_var=True, name="climb")]
pb = xa.Problem(param, points, objective="cd",
                flap_hinge=(0.75, 0.5), flap_bounds=(-5.0, 15.0))
pb.add_constraint("max_thickness", lower=0.12)
pb.add_constraint("cm", point=0, lower=-0.08)
pb.add_constraint("cpmin", point=1, lower=-1.6)

res = xa.optimize(pb, maxiter=40)
print("deflections:", pb.flap_angles(res.dv))
plot_result(pb, res, fig("c10.png"))
save("c10", **summary(pb, res))
```

| point | $C_D$ initial | $C_D$ optimized | change | $\alpha$ (°) | $C_{p,\min}$ | $C_M$ | $x_{tr}$ upper | flap (°) | $C_{mh}$ |
|---|---|---|---|---|---|---|---|---|---|
| cruise | 0.005632 | **0.004774** | −15 % | 0.67 → 1.39 | −0.62 → −0.70 | −0.051 → −0.018 | 0.61 → 0.66 | −1.42 | 0.0032 → −0.0054 |
| climb | 0.010901 | **0.007140** | −35 % | 7.15 → 4.41 | −2.97 → −1.41 | −0.046 → −0.085 | 0.11 → 0.50 | 3.39 | 0.0046 → 0.0017 |

The optimizer deflects the flap by 3.39° in climb and by −1.42° in cruise. $C_D$ decreases by −35 % in climb and by −15 % in cruise (40 iterations, 11 s in series). In climb, $\alpha$ goes from 7.15° to 4.41°: the flap carries part of the lift, the LE suction peak disappears ($C_{p,\min}$ from −2.97 to −1.41) and the upper-side transition moves back.

| constraint | lower bound | initial value | optimized value |
|---|---|---|---|
| `max_thickness` | 0.12 | 0.1213 | 0.12 |
| `cm@cruise` | −0.08 | −0.05075 | −0.01819 |
| `cpmin@climb` | −1.6 | −2.973 | −1.407 |

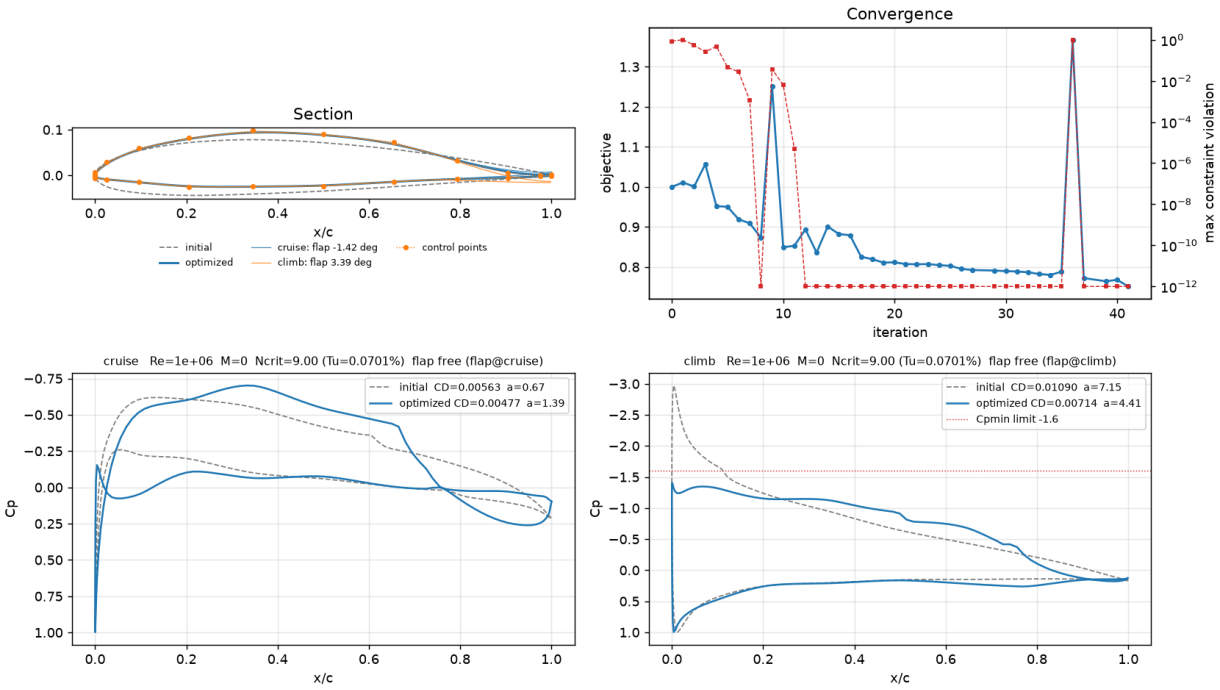



Figure 3: Example c10. Top left: the undeflected section (initial and optimized) and the section seen by each point; bottom: initial and optimized $C_p$, plotted on the deflected nodes.

## 9.6 Example: limited hinge moment

The actuator imposes $|C_{mh}| \leq 0.001$ in climb, several times less than the value obtained in the previous example (0.0017):

guide/cases/c11_flap_hinge.py

```
from _guide import save, fig, summary
import xfoilad as xa
from xfoilad.plotting import plot_result

# Same problem as c10, with a limited hinge moment in climb
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=12)
points = [xa.OperatingPoint(cl=0.3, re=1e6, flap=0.0, flap_var=True, name="cruise"),
          xa.OperatingPoint(cl=1.0, re=1e6, flap=0.0, flap_var=True, name="climb")]
pb = xa.Problem(param, points, objective="cd",
                flap_hinge=(0.75, 0.5), flap_bounds=(-5.0, 15.0))
pb.add_constraint("max_thickness", lower=0.12)
pb.add_constraint("cm", point=0, lower=-0.08)
pb.add_constraint("cpmin", point=1, lower=-1.6)
pb.add_constraint("cmh", point=1, lower=-0.001, upper=0.001)      # limited actuator

res = xa.optimize(pb, maxiter=40)
plot_result(pb, res, fig("c11.png"))
save("c11", **summary(pb, res))
```

| point | $C_D$ initial | $C_D$ optimized | change | $\alpha$ (°) | $C_{p,\min}$ | $C_M$ | $x_{tr}$ upper | flap (°) | $C_{mh}$ |
|---|---|---|---|---|---|---|---|---|---|
| cruise | 0.005632 | **0.005678** | 1 % | 0.67 → 1.44 | −0.62 → −0.71 | −0.051 → −0.013 | 0.61 → 0.60 | −0.08 | 0.0032 → −0.0075 |
| climb | 0.010901 | **0.007274** | −33 % | 7.15 → 5.37 | −2.97 → −1.59 | −0.046 → −0.068 | 0.11 → 0.49 | 1.54 | 0.0046 → 0.0009 |

The hinge moment in climb becomes 0.0009, within the bound. It is reduced partly by deflecting less (1.54° against 3.39° in c10) and partly by changing the shape: on the figure, the upper- and lower-side $C_p$ cross near $x \approx 0.9$ in climb, and the tip of the flap, loaded in the opposite direction, balances the moment about the hinge (aerodynamic balance through the shape).

The limited hinge moment costs drag: $C_D$ 0.007274 against 0.007140 in climb, 0.005678 against 0.004774 in cruise. Both optimizations stopped at the limit of 40 iterations, and the problem is not convex: the constraint, violated at the start (0.0046), leads SLSQP along a different path (the spikes at the end of the convergence history are trial designs where the flow changes regime). A fair comparison of two optimizations therefore requires running them to convergence, or starting from several initial designs.

| constraint | lower bound | initial value | optimized value |
|---|---|---|---|
| `max_thickness` | 0.12 | 0.1213 | 0.131 |
| `cm@cruise` | −0.08 | −0.05075 | −0.01325 |
| `cpmin@climb` | −1.6 | −2.973 | −1.592 |
| `cmh@climb` | −0.001 (upper 0.001) | 0.004592 | 0.0008937 |

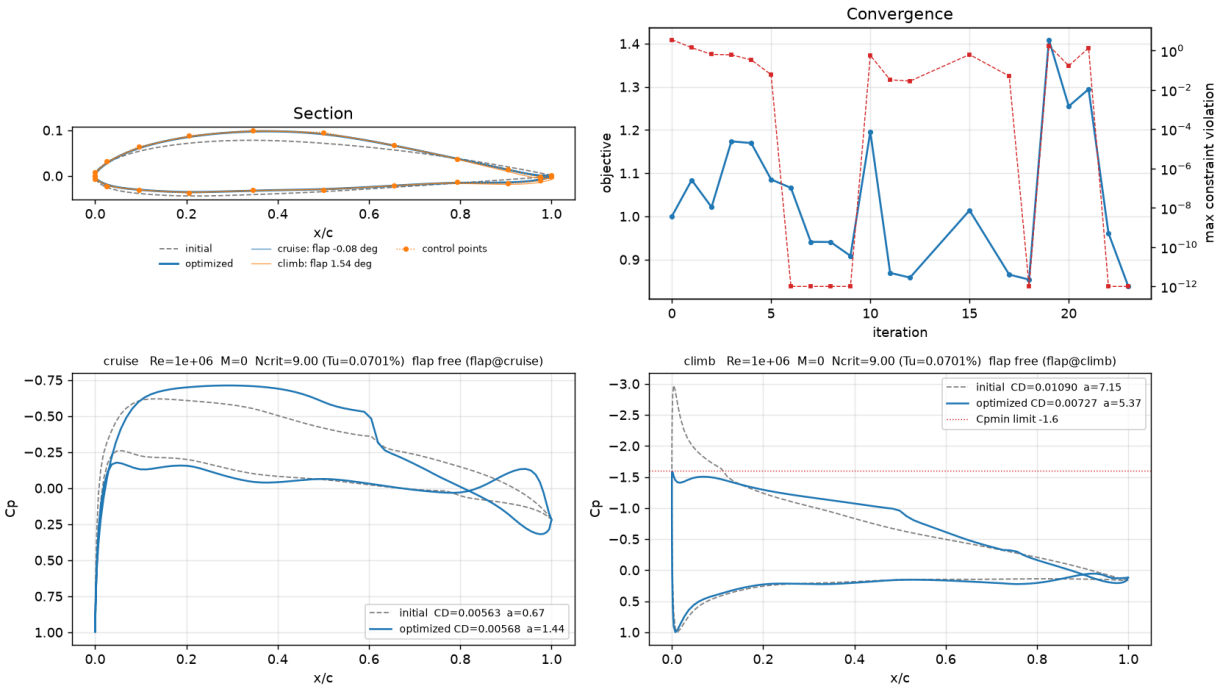



Figure 4: Example c11: same problem as c10, with $|C_{mh}| \leq 0.001$ in climb.

# 10 Shape optimization examples

## 10.1 One point, cavitation constraint

Minimum CD at $C_L = 0.5$ ($\mathrm{Re} = 10^6$), with $t/c \geq 0.10$ and $C_{p,\min} \geq -0.7$ ($\sigma = 0.7$). The initial airfoil is at −0.91 and violates the constraint:

`guide/cases/c05_single_point.py`

```python
from _guide import save, fig, summary
import xfoilad as xa
from xfoilad.plotting import plot_result

# One point: minimum CD at CL = 0.5 with a cavitation constraint (Cpmin >= -0.7)
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=16)            # 27 variables
pb = xa.Problem(param, [xa.OperatingPoint(cl=0.5, re=1e6)], objective="cd")
pb.add_constraint("max_thickness", lower=0.10)
pb.add_constraint("cpmin", point=0, lower=-0.7)

res = xa.optimize(pb, maxiter=45)
xa.write_dat(fig("c05_optimized.dat"), *pb.coordinates(res.dv), "c05 optimized")
plot_result(pb, res, fig("c05.png"))
save("c05", **summary(pb, res))
```

| point | $C_D$ initial | $C_D$ optimized | change | $\alpha$ (°) | $C_{p,\min}$ | $C_M$ | $x_{tr}$ upper |
|---|---|---|---|---|---|---|---|
| cl=0.5 | 0.006012 | **0.004094** | −32 % | 2.48 → 1.45 | −0.91 → −0.60 | −0.049 → −0.092 | 0.50 → 0.81 |

$C_D$ decreases by −32 % (45 iterations, 80 evaluations, 16 s in series) thanks to the rearward move of the upper-side transition (from 0.50 to 0.81) and to a flatter $C_p$ distribution without leading-edge peak: the cavitation constraint, violated at the start, is satisfied with some margin ($C_{p,\min}$ =-0.60).

| constraint | lower bound | initial value | optimized value |
|---|---|---|---|
| `max_thickness` | 0.1 | 0.1214 | 0.1 |
| `cpmin@cl=0.5` | −0.7 | −0.9151 | −0.6291 |

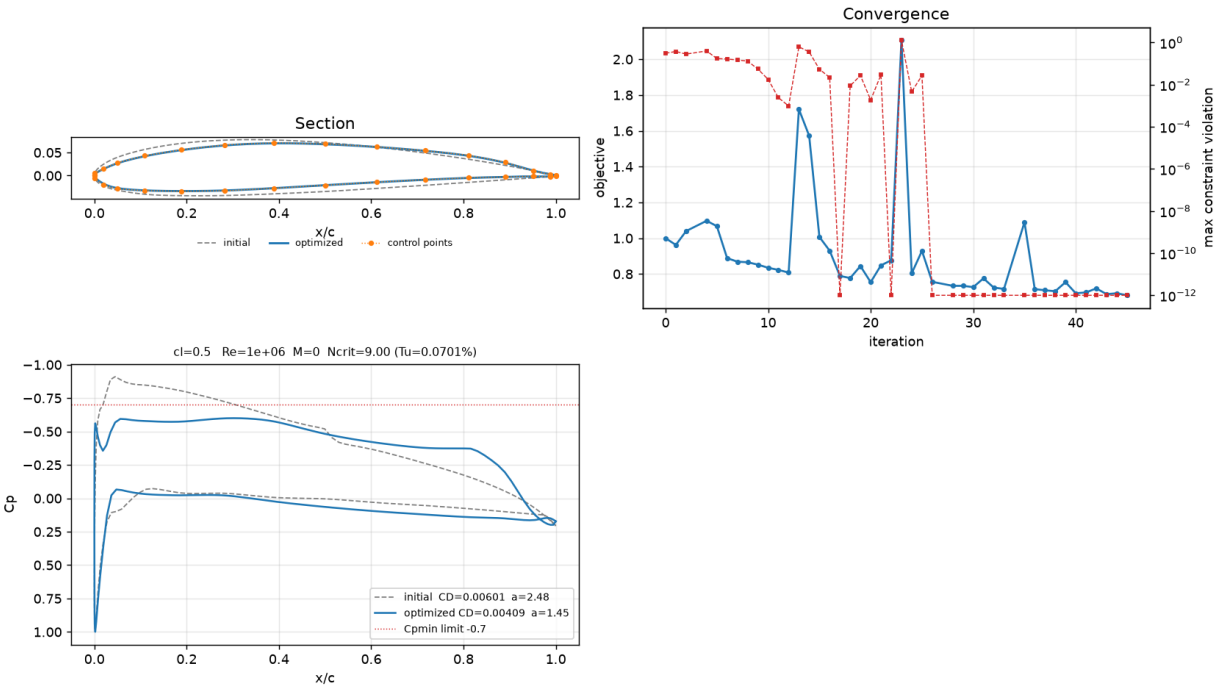



Figure 5: Example c05: section, convergence (normalized objective and maximum constraint violation) and $C_p$.

## 10.2 Three points, structural constraints

Three weighted $C_L$, a cavitation margin and a maximum angle at the high-$C_L$ point, a spar box, the initial bending inertia, a minimum area and a minimum LE radius:

guide/cases/c06_multipoint.py

```python
from _guide import save, fig, summary
import xfoilad as xa
from xfoilad.plotting import plot_result

# Three weighted points, hydrodynamic and structural constraints
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)            # 15 variables
points = [xa.OperatingPoint(cl=0.3, re=8e5, weight=1.0, name="cruise"),
          xa.OperatingPoint(cl=0.6, re=8e5, weight=1.0, name="climb"),
          xa.OperatingPoint(cl=0.9, re=6e5, weight=0.5, name="high CL")]
pb = xa.Problem(param, points, objective="cd", dv_bounds=0.04)
I0 = xa.geomfuncs.inertia(param, param.dv0)[0][0]                   # initial inertia
pb.add_constraint("cpmin", point=2, lower=-1.2)                    # cavitation margin
pb.add_constraint("alpha", point=2, upper=7.0)
pb.add_constraint("box", x_range=(0.15, 0.55), y_range=(-0.035, 0.045))   # spar box
pb.add_constraint("inertia", axis="xx", lower=I0)
pb.add_constraint("area", lower=0.075)
pb.add_constraint("le_radius", lower=0.006)

res = xa.optimize(pb, maxiter=50)
plot_result(pb, res, fig("c06.png"))
save("c06", I0=I0, **summary(pb, res))
```

| point | $C_D$ initial | $C_D$ optimized | change | $\alpha$ (°) | $C_{p,\min}$ | $C_M$ | $x_{tr}$ upper |
|---|---|---|---|---|---|---|---|
| cruise | 0.005699 | **0.005653** | −1 % | 0.69 → −1.77 | −0.63 → −0.54 | −0.050 → −0.121 | 0.65 → 0.85 |
| climb | 0.006646 | **0.005858** | −12 % | 2.99 → 0.70 | −1.01 → −0.73 | −0.058 → −0.127 | 0.49 → 0.75 |
| high CL | 0.010188 | **0.008062** | −21 % | 6.14 → 3.55 | −2.35 → −1.19 | −0.048 → −0.124 | 0.24 → 0.63 |

All the constraints are satisfied (50 iterations, 38 s in series). Without a constraint on $C_M$, the optimizer loads the rear of the airfoil: $C_M$ goes from −0.050 to −0.121 in cruise. If the pitching moment matters (loads on the mast of a foil, setting torque), add `add_constraint("cm", ...)`.

| constraint | lower bound | initial value | optimized value |
|---|---|---|---|
| `cpmin@high CL` | −1.2 | −2.346 | −1.2 |
| `alpha@high CL` | — (upper 7) | 6.137 | 3.548 |
| `box` | 0 | min −0.003601 | min 0 |
| `inertia` | 6.99323e-05 | 6.993e-05 | 8.61e-05 |
| `area` | 0.075 | 0.08225 | 0.0864 |
| `le_radius` | 0.006 | 0.01604 | 0.006851 |

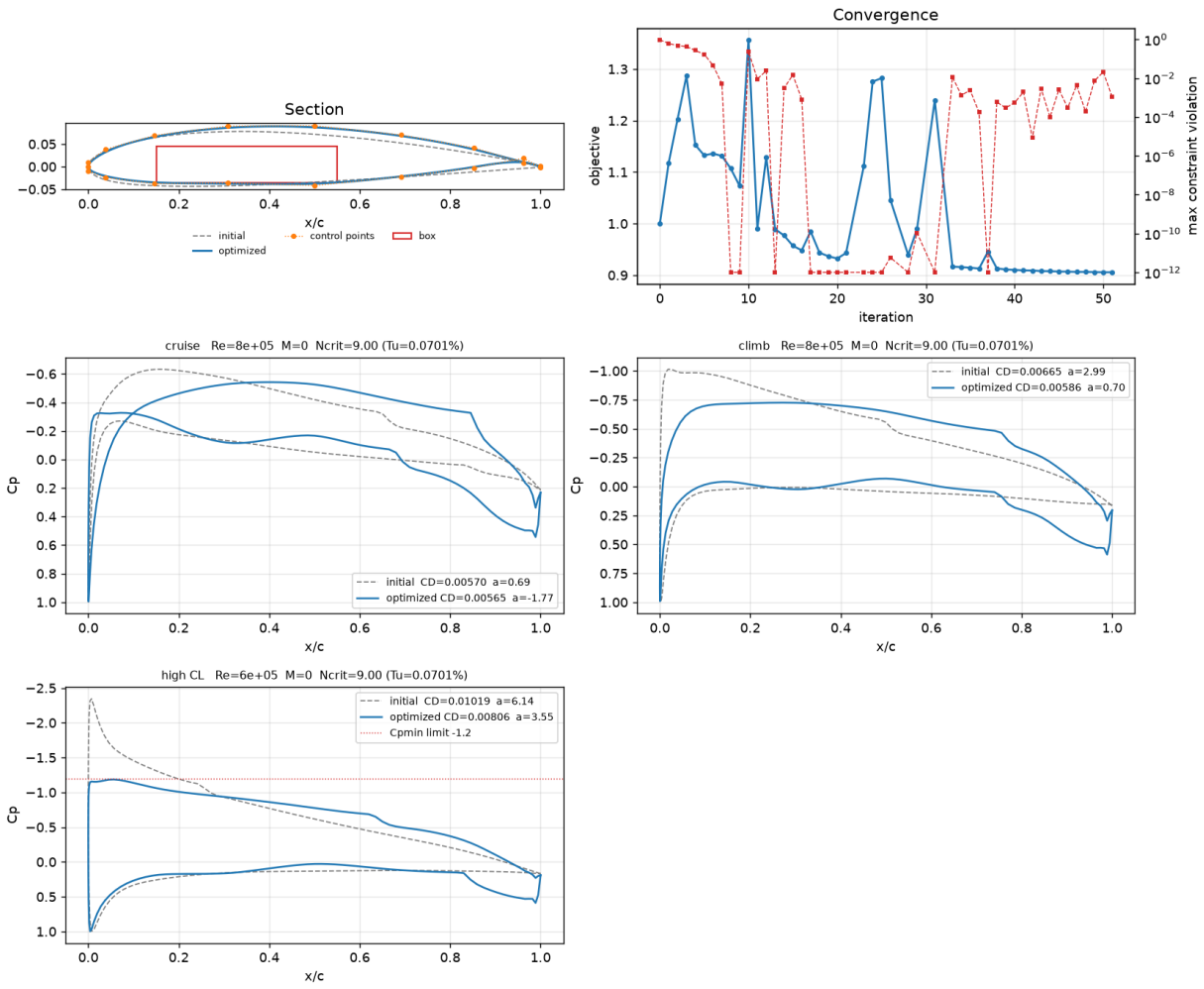



Figure 6: Example c06: the spar box is drawn in red.

### 10.3 Many variables

With 50 control points per side (95 variables, 299 nodes), the cost of an iteration is that of one adjoint pass per function, as with 15 variables. The shape must however be regularized: with so many degrees of freedom, SLSQP creates short-wavelength undulations that distort the $C_p$. The `smoothness` constraint bounds, on each side, the second differences of the displacement of the control ordinates $\delta_k = y_k(\mathbf{d}) - y_k(\mathbf{d}_0)$:

$$-s \;\le\; \delta_{k-1} - 2\delta_k + \delta_{k+1} \;\le\; s. \tag{13}$$

It is linear (exact and constant Jacobian) and zero at the start. Order of magnitude: $s \approx 10^{-3}$ for $n_{ctrl} = 50$; reduce $s$ if the shape undulates, increase it if the optimizer is restrained. Also reduce `dv_bounds` (0.02) and `dv_scale` (0.002).

guide/cases/c08_many_dv.py

```
from _guide import save, fig, summary
import xfoilad as xa
from xfoilad.plotting import plot_result

# 95 shape variables: same gradient cost as with 15
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=50)
pb = xa.Problem(param, [xa.OperatingPoint(cl=0.5, re=1e6, tu=0.1)], objective="cd",
    dv_bounds=0.02)
```

```
pb.add_constraint("max_thickness", lower=0.12)
pb.add_constraint("cpmin", point=0, lower=-1.0)
pb.add_constraint("smoothness", lower=-1e-3, upper=1e-3)          # no waviness

res = xa.optimize(pb, maxiter=60, dv_scale=0.002)
plot_result(pb, res, fig("c08.png"))
save("c08", nodes=param.n_nodes, **summary(pb, res))
```

| point | $C_D$ initial | $C_D$ optimized | change | $\alpha$ (°) | $C_{p,\min}$ | $C_M$ | $x_{tr}$ upper |
|---|---|---|---|---|---|---|---|
| cl=0.5 | 0.006069 | **0.004870** | −20 % | 2.51 → 2.53 | −0.87 → −0.88 | −0.048 → −0.048 | 0.49 → 0.65 |

$C_D$ decreases by −20 % in 60 iterations (33 s in series).

| constraint | lower bound | initial value | optimized value |
|---|---|---|---|
| `max_thickness` | 0.12 | 0.1213 | 0.1202 |
| `cpmin@cl=0.5` | −1 | −0.898 | −0.8927 |
| `smoothness` | −0.001 (upper 0.001) | min 0 | min −0.001 |

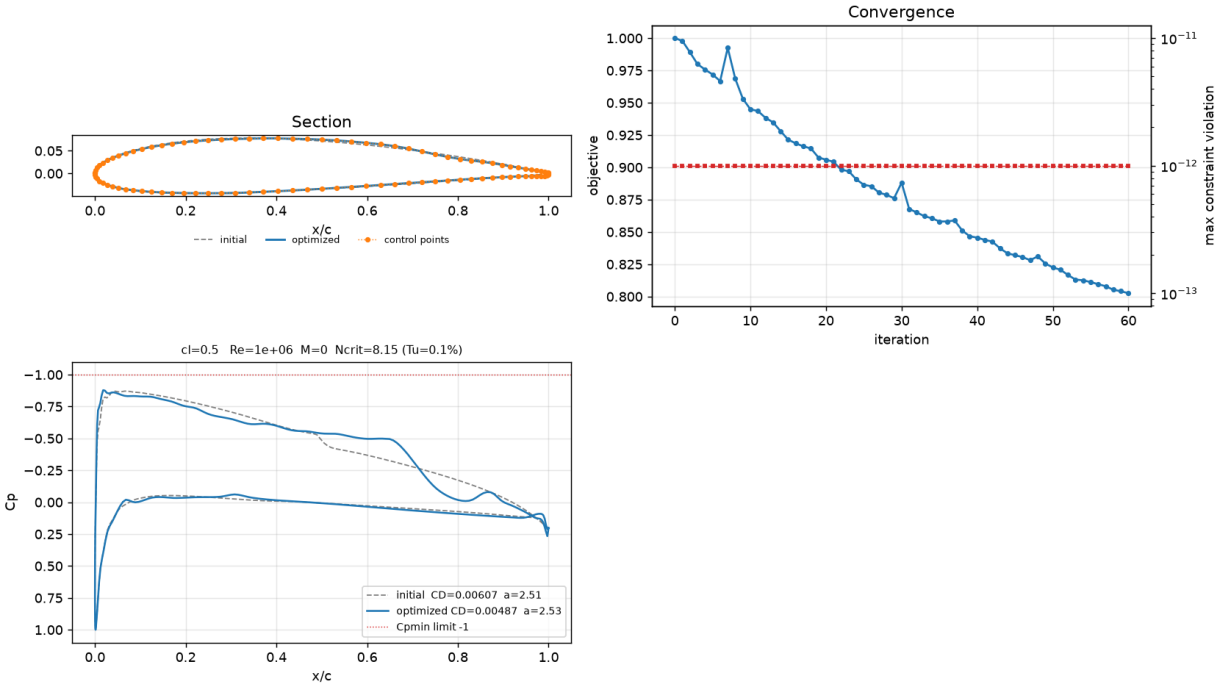



Figure 7: Example c08: 95 shape variables and smoothness constraint.

## 10.4 Symmetric section and skin-friction constraint

Rudder: symmetric section (8 variables), minimum $C_D$ on course ($C_L = 0.2$) and in a turn ($C_L = 0.4$, weight 0.5), and a manoeuvre point ($C_L = 0.9$, Re $= 6 \cdot 10^5$, $T_u = 0.3$ %) that does not count in the objective but carries $\alpha \le 10°$ and, depending on the variant, $C_f \ge 0$. The three variants start from the NACA 0012:

`guide/cases/c14_symmetric_separation.py`

```
from _guide import save, fig, summary
import numpy as np
import xfoilad as xa
```

```
from xfoilad.problem import cf_mask
from xfoilad.plotting import plot_result

# Rudder: symmetric section, minimum CD on course and in a turn; a manoeuvre
# point (CL = 0.9) outside the objective carries the constraint Cf >= 0


def build(cfmin, cfmin_xrange=(0.0, 1.0)):
    param = xa.BSplineAirfoil.from_naca("0012", n_ctrl=10, symmetric=True)      # 8
    variables
    points = [xa.OperatingPoint(cl=0.2, re=1e6, tu=0.1, name="course"),
              xa.OperatingPoint(cl=0.4, re=1e6, tu=0.1, name="turn", weight=0.5),
              xa.OperatingPoint(cl=0.9, re=6e5, tu=0.3, name="manoeuvre", in_objective=
    False)]
    pb = xa.Problem(param, points, objective="cd", cfmin_xrange=cfmin_xrange)
    if cfmin:
        pb.add_constraint("cfmin", point="manoeuvre", lower=0.0)
    pb.add_constraint("alpha", point="manoeuvre", upper=10.0)
    pb.add_constraint("max_thickness", lower=0.12)
    pb.add_constraint("le_radius", lower=0.008)
    return pb


out = {}
for tag, cfmin, xr in (("free", False, (0.0, 1.0)), ("cf_all", True, (0.0, 1.0)), ("
    cf_x01", True, (0.1, 1.0))):
    print(f"\n===== {tag} =====")
    pb = build(cfmin, xr)
    res = xa.optimize(pb, maxiter=40)
    print(pb.report(res.dv, "=== optimized ==="))
    ev = pb.evaluate(res.dv, gradients=False)
    r = ev["results"][2]
    sep = r.x[r.cf < 0]                         # nodes where the flow is separated (whole
     section)
    out[tag] = dict(summary(pb, res), cfmin_all=float(r.cf[cf_mask(r)].min()),
                    sep_x=sep.tolist(), x=r.x, cf=r.cf)
    if tag == "cf_x01":
        plot_result(pb, res, fig("c14.png"))
r0 = build(False).evaluate(build(False).dv0, gradients=False)["results"][2]
out["initial"] = dict(cfmin_all=float(r0.cf[cf_mask(r0)].min()), sep_x=r0.x[r0.cf < 0].
    tolist(), x=r0.x, cf=r0.cf)

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
fig_, ax = plt.subplots(figsize=(8, 3.2))
for tag, lab, st in (("initial", "NACA 0012", ":"), ("free", "unconstrained", "-"),
                     ("cf_all", r"$C_f\geq0$ everywhere", "-"), ("cf_x01", r"$C_f\geq0$
    for $x>0.1$", "--")):
    x, cf = np.array(out[tag]["x"]), np.array(out[tag]["cf"])
    ile = int(np.argmin(x))
    ax.plot(x[:ile + 1], 1e3 * cf[:ile + 1], st, lw=1.2, label=lab)    # upper side (
    suction side)
ax.axhline(0.0, color="k", lw=0.6)
ax.set_xlabel("x/c (upper side)")
ax.set_ylabel(r"$10^3\,C_f$")
ax.set_ylim(-4, 8)
ax.legend(fontsize=8, ncol=2)
ax.set_title("manoeuvre point, CL = 0.9")
```

```
fig_.tight_layout()
fig_.savefig(fig("c14_cf.pdf"))
save("c14", **out)
```

| variant | $J$ | $C_D$ course | $C_D$ turn | $C_D$ manoeuvre | $C_{p,\min}$ manoeuvre | $x/c$ where $C_f < 0$ (manoeuvre) |
|---|---|---|---|---|---|---|
| NACA 0012 | 1 | 0.00591 | 0.00719 | 0.01383 | −3.97 | 0.027–0.036, 0.974–1.000 |
| unconstrained | 0.896 | 0.00522 | 0.00660 | 0.01688 | −5.82 | 0.004–0.012, 0.904–0.982 |
| $C_f \geq 0$ everywhere | 0.978 | 0.00587 | 0.00680 | 0.01379 | −4.27 | none |
| $C_f \geq 0$ for $x > 0.1$ | 0.905 | 0.00533 | 0.00654 | 0.01658 | −5.69 | 0.004–0.012 |

- **Without a friction constraint**, $J$ decreases by −10.4 %, but the optimizer sharpens the leading edge (the minimum radius becomes active): in manoeuvre, $C_{p,\min}$ becomes −5.82, the transition moves up to the leading edge behind a bubble, and the upper side separates at the trailing edge over 8 % of the chord; $C_D$ increases there by 22 %.
- $C_f \geq 0$ **over the whole section**: no reversed flow left in manoeuvre (bubble included), but the gain on the objective is only −2.2 % (40 iterations, 23 s). The initial section already has a bubble: to suppress it, the optimizer keeps a rounder leading edge ($r_{LE} = 0.0088$, above the bound 0.008), which costs drag on course.
- $C_f \geq 0$ **for** $x > 0.1$: the trailing-edge separation disappears and the objective decreases by −9.5 %, almost as much as without constraint. The laminar leading-edge bubble, which reattaches, is tolerated, and the manoeuvre $C_D$, outside the objective, changes by 20 %. This is generally the right choice to express the absence of stall; if the manoeuvre drag matters, put this point in the objective with a small weight.

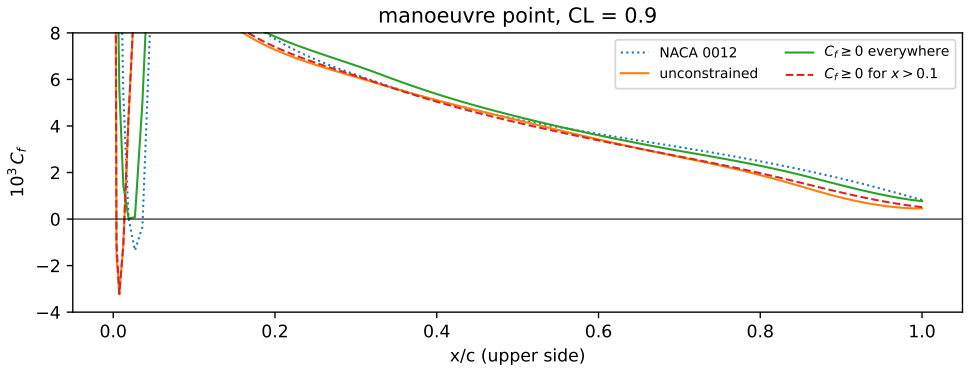



Figure 8: Example c14: $C_f$ on the upper side at the manoeuvre point ($C_L = 0.9$) for the initial section and the three optima.

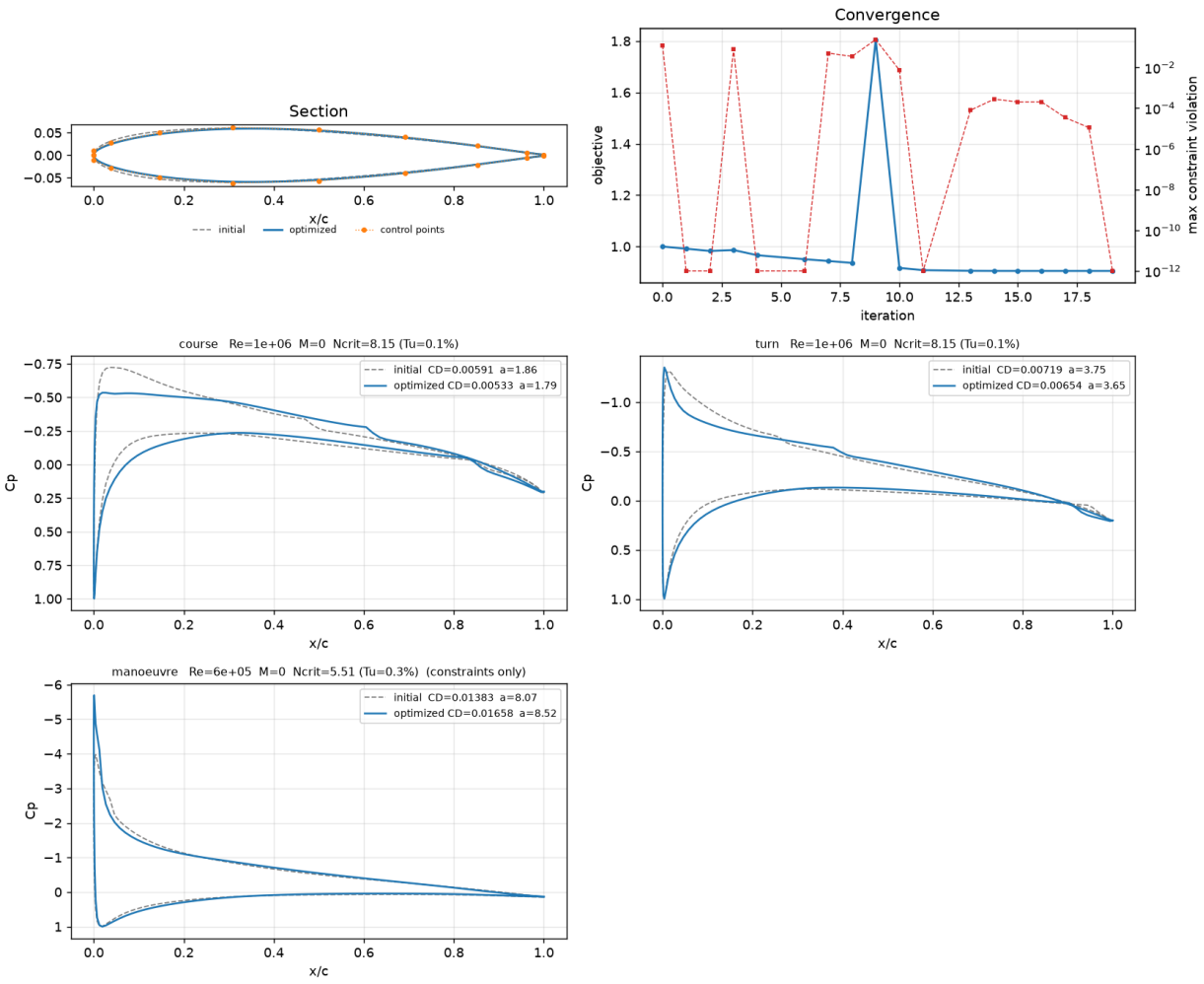



Figure 9: Example c14, variant $C_f \geq 0$ for $x > 0.1$: symmetric section, convergence and $C_p$ of the three points.

## 11 Robust optimization

At sea, the free-stream turbulence is not known: a section optimized for $T_u = 0.07$ % can lose a lot if the transition moves forward. The uncertain parameter is represented by samples $\xi_k$ of probabilities $p_k$, one operating point per sample, and one minimizes

- the **mean** $\sum_k p_k C_{D,k}$: it suffices to give the $p_k$ as the weights (`weight`) of the points;
- or the **superquantile** $\mathrm{CVaR}_\alpha$, the mean of the $(1-\alpha)$ worst cases, with `CVaRProblem`.

`CVaRProblem` uses the differentiable Rockafellar–Uryasev formulation:

$$\mathrm{CVaR}_\alpha = \min_{t,\,s_k} \; t + \frac{1}{1-\alpha} \sum_k p_k s_k \quad \text{subject to} \quad s_k \geq c_k(\mathbf{d}) - t, \;\; s_k \geq 0, \tag{14}$$

with $c_k = C_{D,k}/C_{D,\mathrm{ref}}$. The variables $t$ and $s_k$ are added to the design vector; the constraints of the base problem are kept. Each iteration costs $N$ analyses and $N$ adjoint passes, whatever the number of variables. `ln_uniform_samples(lo, hi, n)` gives the Gauss–Legendre nodes and weights of a log-uniform distribution.

**Example.** $T_u$ log-uniform on $[0.03, 1]$ %, 5 samples, $\mathrm{CVaR}_{0.8}$. The initial shape must converge at *all* the samples: with $[0.05, 1]$ %, XFOIL fails at the start on the sample $T_u = 0.50$ %, and one then changes the range, the number of samples or the initial shape.

guide/cases/c09_robust.py

```
from _guide import save, fig
import numpy as np
import xfoilad as xa

# Uncertain Tu, log-uniform on [0.03, 1] %: 5 Gauss-Legendre samples
tus, p = xa.ln_uniform_samples(0.03, 1.0, 5)
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)
pts = [xa.OperatingPoint(cl=0.5, re=1e6, tu=t, name=f"Tu={t:.3g}%") for t in tus]
base = xa.Problem(param, pts, objective="cd")
base.add_constraint("max_thickness", lower=0.12)
base.add_constraint("cpmin", point="all", lower=-1.0)             # at every sample

rob = xa.CVaRProblem(base, p, alpha=0.8)                          # mean of the 20 %
    worst cases
res = xa.optimize(rob, maxiter=30)
d = res.dv[:base.ndv]                                             # shape variables

cd0 = np.array([r.cd for r in base.evaluate(base.dv0, False)["results"].values()])
cd1 = np.array([r.cd for r in base.evaluate(d, False)["results"].values()])
for t, a, b in zip(tus, cd0, cd1):
    print(f"Tu = {t:.3f} %  CD {a:.5f} -> {b:.5f}")
print("mean", p @ cd0, "->", p @ cd1, "  CVaR_0.8", xa.cvar(cd0, p, 0.8), "->", xa.cvar(
    cd1, p, 0.8))
xa.write_dat(fig("c09_robust.dat"), *base.coordinates(d), "c09 robust")

save("c09", tus=tus, probs=p, cd0=cd0, cd1=cd1, mean0=p @ cd0, mean1=p @ cd1,
     cvar0=xa.cvar(cd0, p, 0.8), cvar1=xa.cvar(cd1, p, 0.8), nit=res.nit, time=res.time,
     history=[h["f"] for h in res.history])
```

| $T_u$ (%) | probability | $C_D$ initial | $C_D$ robust |
|---|---|---|---|
| 0.035 | 0.118 | 0.00579 | 0.00534 |
| 0.067 | 0.239 | 0.00591 | 0.00525 |
| 0.173 | 0.284 | 0.00617 | 0.00518 |
| 0.445 | 0.239 | 0.00680 | 0.00523 |
| 0.848 | 0.118 | 0.00770 | 0.00537 |
| mean | | 0.00640 | 0.00525 |
| $\mathrm{CVaR}_{0.8}$ | | 0.00733 | **0.00536** |

The $\mathrm{CVaR}_{0.8}$ of $C_D$ decreases by $-27$ % and the mean by $-18$ % (30 iterations, 14 s in series). Above all, the robust section is almost insensitive to $T_u$: its $C_D$ varies by less than 4 % over the range, against 33 % for the initial section, whose transition moves forward as $T_u$ increases. A flap deflection shared by all the samples (`flap_var="name"`) is added in the same way: it is then a single setting, robust to $T_u$.

## 11.1 Uncertain incidence at imposed $C_L$

A lifting foil is trimmed to carry the weight: at imposed $C_L$, XFOIL computes the trim incidence $\alpha_{\mathrm{trim}}(\mathbf{d})$. Gusts, the orbital velocity of the waves and pitching then make the incidence seen by the section vary around this trim. This uncertainty is represented by $\alpha$-mode points *attached* to the trim point:

```
xa.OperatingPoint(alpha_from="trim", dalpha=+1.5, re=1e6, tu=0.07)
```

is computed at $\alpha = \alpha_{\mathrm{trim}}(\mathbf{d}) + 1.5°$, where `"trim"` is the name of a point at imposed $C_L$ of the same `Problem`. The trim incidence depends on the shape: the gradient of a function $F$ of the offset

point is

$$\frac{dF}{d\mathbf{d}} = \left.\frac{\partial F}{\partial \mathbf{d}}\right|_{\alpha} + \frac{\partial F}{\partial \alpha}\,\frac{d\alpha_{\mathrm{trim}}}{d\mathbf{d}}, \tag{15}$$

where $\partial F/\partial\alpha$ comes from the same adjoint pass as $\partial F/\partial\mathbf{d}$, and $d\alpha_{\mathrm{trim}}/d\mathbf{d}$ from an adjoint pass of $\alpha$ at the trim point, added automatically. The trim points are computed first, the offset points next. The gradients are checked against finite differences (deviation $\leq 4 \cdot 10^{-7}$).

The distribution of $\Delta\alpha$ is uniform on $[-1.5, 1.5]°$. `uniform_lobatto_samples(lo, hi, n)` gives the Gauss–Lobatto nodes and weights: the bounds are nodes, which allows the cavitation constraint to be imposed there (the suction peak is strongest at the extreme incidences, on the upper side for $+1.5°$ and at the nose of the lower side for $-1.5°$), and, for odd $n$, the central node is the trim itself. To minimize the true mean of $C_D$, the weights are $p_k\, C_{D,k}(\mathbf{d}_0)$, since the objective normalizes each point by its initial value.

guide/cases/c12_robust_alpha.py

```
from _guide import save, fig
import numpy as np
import xfoilad as xa

# Uncertain incidence around the trim (gusts, orbital velocity of the waves):
# alpha = alpha_trim(CL = 0.5) + dalpha, dalpha uniform on [-1.5, 1.5] deg.
# Gauss-Lobatto: the bounds are nodes, the central node is the trim.
das, q = xa.uniform_lobatto_samples(-1.5, 1.5, 5)
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)
pts = [xa.OperatingPoint(cl=0.5, re=1e6, tu=0.07, name="trim")]
pts += [xa.OperatingPoint(alpha_from="trim", dalpha=da, re=1e6, tu=0.07, name=f"{da:+.2f
    } deg")
        for i, da in enumerate(das) if i != len(das) // 2]
w = np.r_[q[len(das) // 2], np.delete(q, len(das) // 2)]          # probabilities, in
    the order of the points

base = xa.Problem(param, pts, objective="cd")
cd0 = np.array([r.cd for r in base.evaluate(base.dv0, False)["results"].values()])
for p, wk, c in zip(pts, w, cd0):            # weights p_k CD_k(d0): the objective is
    the mean of CD
    p.weight = wk * c
pb = xa.Problem(param, pts, objective="cd")
pb.add_constraint("max_thickness", lower=0.12)
for ip, p in enumerate(pts):                 # no cavitation at the extreme incidences
    if p.alpha_from is not None and abs(p.dalpha) == 1.5:
        pb.add_constraint("cpmin", point=ip, lower=-1.0)

res = xa.optimize(pb, maxiter=40)
print(pb.report(res.dv, "optimized"))
ev0, ev1 = pb.evaluate(res.dv0, False), pb.evaluate(res.dv, False)
r0 = [ev0["results"][i] for i in range(len(pts))]
r1 = [ev1["results"][i] for i in range(len(pts))]
xa.write_dat(fig("c12_robust_alpha.dat"), *pb.coordinates(res.dv), "c12 robust in
    incidence")
save("c12", dalpha=[p.dalpha if p.alpha_from is not None else 0.0 for p in pts], probs=w
    ,
     cd0=[r.cd for r in r0], cd1=[r.cd for r in r1], cp0=[r.cpmin for r in r0], cp1=[r.
    cpmin for r in r1],
     cl0=[r.cl for r in r0], cl1=[r.cl for r in r1], alpha0=r0[0].alpha, alpha1=r1[0].
    alpha,
     mean0=float(w @ [r.cd for r in r0]), mean1=float(w @ [r.cd for r in r1]),
     nit=res.nit, time=res.time, history=[h["f"] for h in res.history])
```

| $\Delta\alpha$ (°) | probability | $C_D$ initial | $C_D$ robust | $C_{p,\min}$ initial | $C_{p,\min}$ robust |
|---|---|---|---|---|---|
| $-1.50$ | 0.050 | 0.00550 | 0.00479 | $-0.66$ | $-0.59$ |
| $-0.98$ | 0.272 | 0.00552 | 0.00460 | $-0.72$ | $-0.63$ |
| $+0.00$ | 0.356 | 0.00592 | 0.00460 | $-0.86$ | $-0.72$ |
| $+0.98$ | 0.272 | 0.00656 | 0.00481 | $-1.16$ | $-0.81$ |
| $+1.50$ | 0.050 | 0.00699 | 0.00504 | $-1.35$ | $-0.86$ |
| mean | | 0.00602 | **0.00469** | | |

The mean of $C_D$ over $\Delta\alpha$ changes by $-22$ % (40 iterations, 40 s in series), and the worst $C_{p,\min}$ goes from $-1.35$ to $-0.86$: the robust section keeps a cavitation *bucket* that covers the whole incidence range, whereas an optimization at the trim alone places the suction peak exactly on the bound and cavitates as soon as the incidence departs from it.

### 11.2 Double uncertainty: incidence and turbulence

The two uncertainties are combined as a tensor product: for each sample $T_{u,j}$, a trim point (the trim incidence depends on $T_u$) and its offset points, with probabilities $p_j\, q_i$. With 3 Gauss–Legendre nodes in $\ln T_u$ and 3 Gauss–Lobatto nodes in $\Delta\alpha$, the problem has 9 points; each iteration costs these 9 analyses and as many adjoint passes (plus one per cavitation constraint and one for $\alpha$ at each trim), whatever the number of variables. The CVaR applies in the same way: just pass the $p_j q_i$ to `CVaRProblem`.

guide/cases/c13_robust_double.py

```python
from _guide import save, fig
import numpy as np
import xfoilad as xa

# Double uncertainty: incidence (uniform on trim +- 1.5 deg, Gauss-Lobatto)
# and turbulence (log-uniform on [0.03, 1] %, Gauss-Legendre), independent.
das, q = xa.uniform_lobatto_samples(-1.5, 1.5, 3)                  # -1.5, 0, +1.5 deg
tus, p = xa.ln_uniform_samples(0.03, 1.0, 3)
param = xa.BSplineAirfoil.from_naca("2412", n_ctrl=10)
pts, w = [], []
for t, pt in zip(tus, p):
    trim = f"Tu={t:.3g}%"                    # one trim per turbulence level: alpha_trim
     depends on Tu
    pts.append(xa.OperatingPoint(cl=0.5, re=1e6, tu=t, name=trim))
    w.append(pt * q[1])
    for da, qa in ((das[0], q[0]), (das[2], q[2])):
        pts.append(xa.OperatingPoint(alpha_from=trim, dalpha=da, re=1e6, tu=t, name=f"{
    trim} {da:+g} deg"))
        w.append(pt * qa)
w = np.array(w)

base = xa.Problem(param, pts, objective="cd")
cd0 = np.array([r.cd for r in base.evaluate(base.dv0, False)["results"].values()])
for pk, wk, c in zip(pts, w, cd0):
    pk.weight = wk * c                       # objective: mean of CD over the joint
    distribution
pb = xa.Problem(param, pts, objective="cd")
pb.add_constraint("max_thickness", lower=0.12)
for ip, pk in enumerate(pts):
    if pk.alpha_from is not None:            # no cavitation at the extreme incidences,
    at every Tu
        pb.add_constraint("cpmin", point=ip, lower=-1.0)

res = xa.optimize(pb, maxiter=40)
```

```
print(pb.report(res.dv, "optimized"))
ev0, ev1 = pb.evaluate(res.dv0, False), pb.evaluate(res.dv, False)
r0 = [ev0["results"][i] for i in range(len(pts))]
r1 = [ev1["results"][i] for i in range(len(pts))]
xa.write_dat(fig("c13_robust_double.dat"), *pb.coordinates(res.dv), "c13 robust in
    incidence and Tu")
save("c13", names=[pk.name for pk in pts], tu=[pk.tu for pk in pts],
     dalpha=[pk.dalpha if pk.alpha_from is not None else 0.0 for pk in pts], probs=w,
     cd0=[r.cd for r in r0], cd1=[r.cd for r in r1], cp0=[r.cpmin for r in r0], cp1=[r.
    cpmin for r in r1],
     mean0=float(w @ [r.cd for r in r0]), mean1=float(w @ [r.cd for r in r1]),
     nit=res.nit, time=res.time, npoints=len(pts), history=[h["f"] for h in res.history
    ])
```

| $T_u$ (%) | $\Delta\alpha$ (°) | probability | $C_D$ initial | $C_D$ robust | $C_{p,\min}$ initial | $C_{p,\min}$ robust |
|---|---|---|---|---|---|---|
| 0.045 | +0.0 | 0.185 | 0.00582 | 0.00494 | −0.86 | −0.73 |
| 0.045 | −1.5 | 0.046 | 0.00529 | 0.00490 | −0.65 | −0.61 |
| 0.045 | +1.5 | 0.046 | 0.00681 | 0.00508 | −1.33 | −0.85 |
| 0.173 | +0.0 | 0.296 | 0.00617 | 0.00482 | −0.87 | −0.73 |
| 0.173 | −1.5 | 0.074 | 0.00612 | 0.00475 | −0.68 | −0.61 |
| 0.173 | +1.5 | 0.074 | 0.00743 | 0.00517 | −1.38 | −0.86 |
| 0.674 | +0.0 | 0.185 | 0.00731 | 0.00528 | −0.87 | −0.73 |
| 0.674 | −1.5 | 0.046 | 0.00760 | 0.00539 | −0.67 | −0.62 |
| 0.674 | +1.5 | 0.046 | 0.00817 | 0.00582 | −1.28 | −0.86 |
| mean | | | 0.00656 | **0.00504** | | |

The joint mean of $C_D$ changes by −23 % (40 iterations, 71 s), and the worst $C_{p,\min}$ goes from −1.38 to −0.86.

### 11.3 Samples per condition and CVaR per group

`incidence_samples(points, dalpha=1.5, n=5, fold_zero=False)` replaces each point by the Gauss–Lobatto nodes of an incidence uniform on $[-\Delta\alpha, \Delta\alpha]$: a point at imposed $C_L$ gives its trim point (central node) and `alpha_from` points at $\alpha_{\text{trim}} + \delta$; an $\alpha$-mode point gives points at $\alpha + \delta$. With `fold_zero=True` (symmetric section), a zero-lift point is computed only at $\alpha = 0$ and at the positive nodes, $C_D(-\alpha) = C_D(\alpha)$, with the folded probabilities. The weight of each sample is that of the point multiplied by its probability: a `Problem(normalize="global")` therefore minimizes the mean. The function also returns the groups (one per original point); a constraint is placed on each sample of the group.

`CVaRProblem(base, probs=None, alpha=0.9, groups=None, start=None)` accepts these groups: the objective becomes $\sum_g w_g \,\mathrm{CVaR}_\alpha(f_k, k \in g)$, one CVaR per condition, $f_k$ being the objective function of the base problem (`drag` included, the chord then being in $f_k$). `start` gives the starting shape, for instance the optimum of the mean: starting directly from the initial section can make the CVaR stall (example below).

```
pts, groups = xa.incidence_samples(conditions, dalpha=1.5, n=5, fold_zero=True)
base = xa.Problem(param, pts, objective="drag", normalize="global",
                  chord=xa.ChordFromInertia(I_ref), nu=1.19e-6)
for g in groups:
    if not g["objective"]:                    # constraint condition: on its 5 samples
        for ip in g["idx"]:
            base.add_constraint("cfmin", point=ip, lower=0.0)
res_mean = xa.optimize(base, maxiter=60)                                  # mean
cv = xa.CVaRProblem(base, alpha=0.9, groups=[g for g in groups if g["objective"]],
                    start=res_mean.dv)
```

```
res_cvar = xa.optimize(cv, maxiter=60)                                    # CVaR per
    condition
```

**Smoothed transition (`ncrit_samples`).** The $e^N$ transition jumps from one boundary-layer station to the next when the shape varies: $C_D$ is only piecewise differentiable. `ncrit_samples(points, spread=1.0, n=3)` replaces each point by $n$ samples of $N_{crit}$ uniform on $N_{crit}\pm$`spread`; the jumps occur at different shapes for each $N_{crit}$, and their mean is smoother. On a NACA 2412 at $C_L = 0.5$, the largest second difference of $C_D$ along a design variable is divided by 3 (from $1.6 \cdot 10^{-5}$ to $5.5 \cdot 10^{-6}$), the nominal value changing by only $4 \cdot 10^{-6}$. But the optimum is not systematically better: on case c05 (16 control points, 45 iterations), the final nominal $C_D$ goes from 0.004130 to 0.004084 starting from the NACA 2412, and from 0.004230 to 0.004360 starting from the NACA 4412, for three times the cost. It is an option, not a default setting. A transition spread within the boundary-layer equations themselves (intermittency) remains outside the current scope.

## 12 Experimental validation

The XFOIL core of XFOIL$_{\text{AD}}$ is compared with two sets of measurements on the NACA 0012 at $Re = 6 \cdot 10^6$, taken as they are from the database of the NASA *Turbulence Modeling Resource*:

- **Ladson** (NASA TM 4074, 1988), Langley Low-Turbulence Pressure Tunnel, $M = 0.15$, transition tripped by carborundum strips at $x/c = 0.05$ on both sides (grits 80, 120 and 180);
- **Abbott & von Doenhoff** (*Theory of Wing Sections*), free transition, $M \approx 0$, approximate digitization of the plots of the book.

XFOIL$_{\text{AD}}$ uses the repaneled closed-form NACA section ($2\times150$ panels), $N_{crit} = 9$, and for Ladson $M = 0.15$ and $x_{tr} = 0.05$ (21 incidences from $-4$ to 16°, no failure);

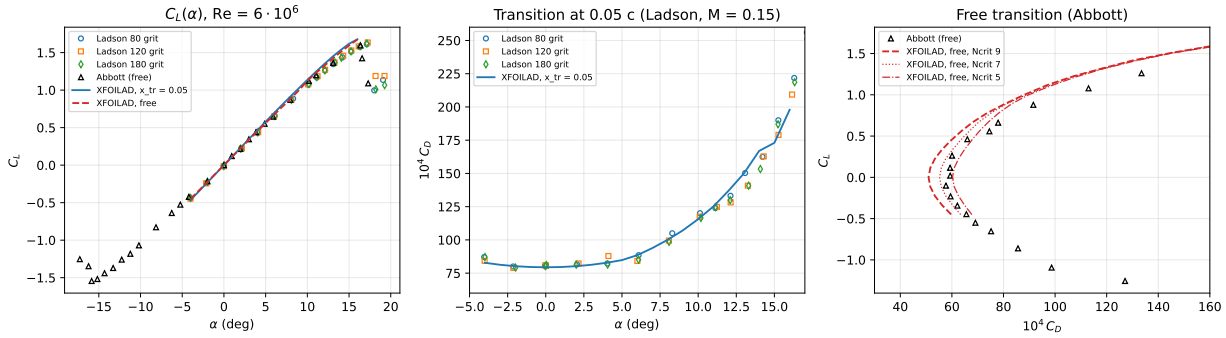



Figure 10: NACA 0012, $Re = 6 \cdot 10^6$: $C_L(\alpha)$, $C_D(\alpha)$ with transition at $0.05\,c$ (Ladson), and polar in free transition (Abbott), with the sensitivity to $N_{crit}$.

| | XFOIL$_{\text{AD}}$ | measurements |
|---|---|---|
| transition at $0.05\,c$: $C_{D,\min}$ | 0.00794 | 0.00789–0.00800 |
| transition at $0.05\,c$: mean deviation on $C_D$ ($\lvert\alpha\rvert \le 8.5°$) | $-1.3$ to $+0.1$ % depending on the grit | |
| transition at $0.05\,c$: $dC_L/d\alpha$ (/°) | 0.115 | 0.108–0.109 |
| free transition: $C_{D,\min}$ | 0.0051 | 0.0058 |
| free transition: $dC_L/d\alpha$ (/°) | 0.112 | 0.106 |

With fixed transition, the drag is reproduced within 1 % on average up to stall, which occurs around 16° in both cases. The lift slope of XFOIL is 6 % higher; Ladson notes the same overestimation by the computational methods of the time, and his wind tunnel was not corrected for wall effects at this level. In free transition, $\text{XFOIL}_{\text{AD}}$ gives a lower drag (13 % on average over $|C_L| \leq 0.9$): it depends directly on the turbulence of the wind tunnel, which is unknown (Ladson reports that it had increased), and which $N_{crit} = 9$ assumes to be very low. Without any fitting, the figure shows the sensitivity: with $N_{crit} = 7$ ($T_u \approx 0.16$ %), the mean deviation is $-8$ %; with $N_{crit} = 5$ ($T_u \approx 0.37$ %), $-1.5$ %. In hydrodynamics, $N_{crit}$ (or $T_u$) is therefore the parameter to set with care, or to treat as uncertain (Section 11).

# 13 Python API reference

**`BSplineAirfoil(n_ctrl=8, degree=3, te=0.0025, n_side=None, x_ctrl=None, dv0=None, le_cluster=0.08, te_cluster=0.35, symmetric=False, le_node=None)`**
Constructors `from_naca(code, n_ctrl)`, `from_dat(path, n_ctrl)`, `from_coordinates(x, y, n_ctrl, te=None)`. Methods `coordinates(dv)`, `jacobian()`, `upper_op(xs)`, `lower_op(xs)`, `le_radius(dv)`, `control_polygon(dv)`, `fit(x, y)`; attributes `dv0`, `ndv`, `n_nodes`, `names`, `symmetric`, `le_node` (node at the leading edge; `None`: yes, except for a symmetric section); `n_dv(n_ctrl, symmetric)` gives the number of variables.

**`CSTAirfoil(order=6, te=0.0025, n_side=80, symmetric=False, le_node=None)`**
same constructors, with `order`.

**`FlapDeflection(x, y, delta, x_hinge, y_hinge, blend=0.03)`**
deflected nodes `.x`, `.y`; `pullback(gx, gy)` $\rightarrow (\partial J/\partial x,\ \partial J/\partial y,\ \partial J/\partial y_h,\ \partial J/\partial \delta)$.

**`OperatingPoint(...)`**
see Section 7; `speed` (m/s) gives $Re = Vc/\nu$ with a chord model (§7.3).

**`cavitation_number(speed, depth=0, p_atm=101325, p_v=2340, rho=1025, g=9.81)`**
$\sigma = (p_{atm} + \rho g\, z - p_v)/(\frac{1}{2}\rho V^2)$ at depth $z$ (m) and speed $V$ (m/s), sea water around 20 °C by default.

**`FixedChord(c)`, `ChordFromInertia(I, axis="xx")`, `ChordFromModulus(W)`, `KNOT`**
dimensional chord models (module `sizing`, §7.3); `model(param, d)` $\rightarrow (c,\ \partial c/\partial d)$.

**`Problem(param, points, objective="cd", dv_bounds=0.04, kiter=4, tol=1e-5, solver=None, cpmin_rho=80, cpmin_xrange=(0, 0.98), verbose=True, n_workers=None, flap_hinge=(0.75, 0.5), flap_bounds=(-10, 15), flap_blend=0.03, flap_scale=1.0, cfmin_rho=5e4, cfmin_xrange=(0, 1), cfmin_side="both", cf_kiter=6, chord=None, nu=None, normalize="point", warm_start=False)`**
`dv_bounds`: scalar ($\mathbf{d}_0 \pm$) or pair (lb, ub) on the shape. `kiter`: Newton iterations replayed by the adjoint. `tol`: rms tolerance of the XFOIL Newton method ($10^{-4}$ in XFOIL, $10^{-5}$ here for clean gradients). `n_workers`: `None` = `XFOILAD_WORKERS` or half of the cores, 1 = serial. `solver`: an existing `XFoilAD` (with `n_workers=1`). Methods: `add_constraint(kind, lower, upper, equals, point, scale, name, **options)`, `evaluate(dv, gradients=True)` (dictionary `f`, `grad`, `cvals`, `cgrads`, `results`, `flaps`, `x`, `y`, `chord`, `re`, `fk`, `fk_grad`), `report(dv)`, `coordinates(dv, point=None)`, `chord(dv)`, `shape_dv(dv)`, `flap_angles(dv)`, `hinge_y(dv)`, `hinge_moments(ev)`, `dv_scales(dv_scale)`, `sensitivities(dv=None, functions=("cl", "cd", "cp"), points=None, method="adjoint")` (list of dictionaries `name`, `mode`, `x`, `y`, `values`, `dx`, `dy`, `dop`, `ddv`, §3.3), `write_sensitivities(out_dir, dv=None, functions, points, prefix="sens", method="adjoint")`; attributes `dv0`, `ndv`, `n_shape`, `names`, `flap_vars`, `objective_points` (indices of the points of the objective), `lb`, `ub`, `hydro` (the `HydroEvaluator`).

`optimize(problem, maxiter=60, ftol=1e-6, dv_scale=0.01, verbose=True, callback=None, method="slsqp", feas_tol=1e-4, **ipopt_kw)`
returns an `OptResult`: `dv0`, `dv`, `success`, `message`, `nit`, `history`, `time`, `to_json(path, problem)`. The history gives, per iteration, the objective, the maximum violation and, per point, $C_L$, $C_D$, $C_M$, $\alpha$, $C_{p,\min}$ (plus the deflection and $C_{mh}$ of the points with a flap). With `method="ipopt"`, `ipopt_kw` accepts `dll` and `options`, and the result also has `nfev` and `nfail`.

`CVaRProblem(base, probs=None, alpha=0.9, t_max=5.0, groups=None, start=None)`
Sections 11 and 11.3; passed directly to `optimize`; `res.dv[:base.ndv]` are the variables of the base problem. The samples are the points of the objective of `base`; its points outside the objective keep their constraints. `groups`: one weighted CVaR per group; `start`: starting shape. `incidence_samples(points, dalpha=1.5, n=5, fold_zero=False)`, `ncrit_samples(points, spread=1.0, n=3)` → (samples, groups), §11.3. `ln_uniform_samples(lo, hi, n)`, `uniform_lobatto_samples(lo, hi, n)` (§11.1), `quantile(values, probs, alpha)`, `cvar(values, probs, alpha)`.

`HydroEvaluator`
robust analyses and adjoint passes of a list of points (`pb.hydro`); `close()` stops the worker processes.

`problem.cf_mask(result, xrange, side)`
nodes selected by `cfmin`.

`problem.hinge_moment(x, y, cp, xh, yh, ops)`
value of $C_{mh}$ and its derivatives (with respect to $C_p$, $x$, $y$, $y_h$); `ops` comes from `problem.hinge_contour(x_undeflected, xh)`.

`XFoilAD()`
low-level access: `set_flow(re, mach, ncrit, xtr, verbose)`, `set_tolerance`, `set_alpha_guess`, `set_stag_frac(f)` (minimum distance of the stagnation point to the nodes, as a fraction of a panel, 0.01), `analyze(x, y, alpha|cl, niter, warm, init=None)` → `Result` (cl, cd, cdf, cdp, cm, alpha, xtr_top, xtr_bot, cp, cpmin, cf, cfmin, converged, x, y, state: `BLState` if converged; `init`: starting state), `adjoint(x, y, alpha|cl, seed, cp_seed, cf_seed, kiter, flow=False)` → ($\partial J/\partial x$, $\partial J/\partial y$, $\partial J/\partial$op) and, with `flow=True`, the dictionary of the derivatives with respect to Re and the $N_{crit}$; `get_state()` → `BLState` (`r`, `i`, `n`, `alpha`, `cl`, `flow`, `copy()`), `set_state(state)` (warm restart). Each `XFoilAD` loads its own copy of the DLL: two instances do not share any state.

`XFoilTangent()`
same methods on the tangent library (`Result.state`, `analyze(..., init=)`, `get_state`, `set_state`, `set_stag_frac` included), plus `tangent(x, y, alpha|cl, dx, dy, dop=0, dflow=(0, 0, 0), kiter=4)` → dictionary of the derivatives of cl, cd, cdf, cm, alpha, cp, cf.

`plotting.plot_result(problem, res, path)`
figure: section and control points (and deflected sections), convergence, $C_p$ of each point.

`write_dat(path, x, y, name)`, `read_dat(path)`, `naca4(code, n)`, `repanel(x, y, n_side)`
coordinate utilities.

# 14 Command line and JSON file

`python -m xfoilad config.json` writes `report.txt`, `history.json`, `initial.dat`, `optimized.dat` (undeflected section), `optimized_point<i>.dat` (deflected section of each point with a flap) and `result.png` in the `output` folder. Example with a flap and a hinge moment, which reproduces example c11:

```
{
  "airfoil": {"naca": "2412"},
  "parametrization": {"type": "bspline", "n_ctrl": 12},
  "objective": "cd",
  "points": [
    {"cl": 0.3, "re": 1e6, "name": "cruise", "flap": 0.0, "flap_var": true},
    {"cl": 1.0, "re": 1e6, "name": "climb", "flap": 0.0, "flap_var": true}
  ],
  "flap": {"hinge": [0.75, 0.5], "bounds": [-5.0, 15.0], "blend": 0.03, "scale": 1.0},
  "constraints": [
    {"kind": "max_thickness", "lower": 0.12},
    {"kind": "cm", "point": 0, "lower": -0.08},
    {"kind": "cpmin", "point": 1, "lower": -1.6},
    {"kind": "cmh", "point": 1, "lower": -0.001, "upper": 0.001}
  ],
  "optimizer": {"maxiter": 40, "dv_bounds": 0.04, "dv_scale": 0.01},
  "output": "results_flap"
}
```

The `"flap"` block gives the hinge (`hinge`), the bounds, the half-width of the ramp (`blend`) and the scale (`scale`); it is optional. The `"optimizer"` block also accepts `kiter`, `tol`, `cpmin_rho`, `cpmin_xrange`, `n_workers`, `ftol`, `cfmin_rho`, `cfmin_xrange`, `cfmin_side`, `cf_kiter`, `warm_start`. The entries `"chord"` (a number, `{"inertia": I, "axis": "xx"}` or `{"modulus": W}`), `"nu"`, `"normalize"` and `"robust_incidence"` (`dalpha`, `n`, `fold_zero`, `measure`: `"mean"` or `"cvar"`, `level`) give the sizing (§7.3) and the robustness in incidence (§11.3); a point accepts `"speed_kn"` (knots) or `"speed"` (m/s), and a constraint placed on a point applies to all its samples. The `"sensitivities"` block accepts `"method": "tangent"`. `"parametrization"` accepts `"type": "cst"`, `order` and `"symmetric": true`; `"airfoil"` accepts `{"dat": "airfoil.dat"}`. A point takes all the keys of `OperatingPoint`, including `"in_objective": false`, and a constraint can designate its point by its name.

Example of a symmetric section with a $C_f \geq 0$ point outside the objective and the export of the sensitivities (variant $x > 0.1$ of example c14):

```
{
  "airfoil": {"naca": "0012"},
  "parametrization": {"type": "bspline", "n_ctrl": 10, "symmetric": true},
  "objective": "cd",
  "points": [
    {"cl": 0.2, "re": 1e6, "tu": 0.1, "name": "course"},
    {"cl": 0.4, "re": 1e6, "tu": 0.1, "name": "turn", "weight": 0.5},
    {"cl": 0.9, "re": 6e5, "tu": 0.3, "name": "manoeuvre", "in_objective": false}
  ],
  "constraints": [
    {"kind": "cfmin", "point": "manoeuvre", "lower": 0.0},
    {"kind": "alpha", "point": "manoeuvre", "upper": 10.0},
    {"kind": "max_thickness", "lower": 0.12},
    {"kind": "le_radius", "lower": 0.008}
  ],
  "optimizer": {"maxiter": 30, "dv_bounds": 0.04, "cfmin_xrange": [0.1, 1.0]},
  "sensitivities": {"functions": ["cl", "cd", "cfmin", "cp"], "designs": ["optimized"],
    "points": ["course"]},
  "output": "results_symmetric"
}
```

The `"sensitivities"` block (optional) gives the functions (`functions`, default `["cl", "cd", "cp"]`), the designs (`designs`: `"initial"` and/or `"optimized"`, default the latter) and the points (`points`, default all of them); the files of `write_sensitivities` go to `<output>/sensitivities_`

`<design>/`. A list alone, `"sensitivities": ["cl", "cd"]`, stands for `functions`.

# 15 Computational cost, cache and parallelism

Orders of magnitude (159 nodes, i7-1260P): analysis 0.01 to 0.016 s, adjoint pass 0.04 s, XFOIL failure 0.1 s (150 iterations then continuation). With 299 nodes, an adjoint pass costs 0.14 s: the cost grows with the number of *nodes*, not with the number of variables. The flap and the hinge moment only add vector operations in Python, and `cmh` one adjoint pass per point where it is requested. A `cfmin` pass costs about 25 % more than a $C_D$ pass. The sensitivities of `"cp"` or `"cf"` cost one pass per node (159 passes, 9 s for example c15): this is a post-processing step, not a function to put in an optimization loop. Their derivatives with respect to the design variables cost one pass per variable in tangent mode (`method="tangent"`, 3.9 s instead of 220 s for 3 points and 15 variables). The derivatives with respect to Re and $N_{crit}$ are free: they come from the same adjoint pass.

- **Cache.** `Problem.evaluate` keeps the last 16 designs, including those where XFOIL failed. SLSQP requests the objective, each constraint and their gradients separately: a design is computed only once.
- **Gradients on demand.** SLSQP and IPOPT evaluate the objective and the constraints without adjoint; the rejected trial points of the line search therefore cost only their analyses. When the gradient of an already evaluated design is requested, the adjoint passes restart from its converged state, without a new analysis.
- `Problem(..., warm_start=True)`: each point first restarts from its converged state at the previous design (then cold start and continuation if this fails). The result then depends on the history, within the convergence tolerance; the gradients remain those of the converged solution. The time gain is modest, the optimum can be slightly worse and a robust optimization can be slower: the option is therefore off by default.
- **Parallelism.** With gradients, each (point, function) pair is an independent task (analysis then adjoint pass), run in a separate process that has its own copy of the library. The results are bit-for-bit identical to those of the serial computation. No `if __name__ == "__main__"` guard is needed. The gain depends on the number of passes per design (about 3× on a problem with 5 passes).
- `kiter=3` makes each pass 14 % cheaper (error $3\cdot10^{-6}$ instead of $4\cdot10^{-9}$), but on the examples SLSQP then follows another path and ends slightly worse: the default stays at 4.

The times quoted were measured in series, on a laptop (i7-1260P) that was running other computations: they are indicative.

# 16 Robustness, limitations and troubleshooting

- **XFOIL non-convergence.** Each point is first run from a cold start. If it fails, an $\alpha$ continuation is attempted (start at 0 then at $\alpha/2$, warm steps of 0.75°, then back to CL mode; for a point defined by `alpha_from`, first from the trim incidence, which has just converged). Before this continuation, $\text{XFOIL}_{\text{AD}}$ attempts a *warm restart*: each point keeps the boundary-layer state of its last convergence (on the previous, neighbouring shape), and the analysis restarts from it (`XFoilAD.get_state`, `set_state`). A shape that converges from a cold start therefore never depends on the history. If everything fails, the objective is penalized and SLSQP backs off (`! XFOIL failed ... -> penalized`). A few isolated failures are normal. A point rescued by a warm restart depends on the stored state, hence on what the `Problem` has evaluated before: evaluating designs with the same `Problem` before

`optimize` can change the optimization path on a multimodal problem. To reproduce a run, evaluate reference designs with a separate `Problem`.

- **Crossed sections.** The parametrization does not prevent the lower side from crossing above the upper side, typically near the trailing edge where the section is thin. XFOIL then returns meaningless results that an optimizer can exploit. A design whose local thickness $y_u(x) - y_l(x)$ becomes negative (`geometry.min_thickness`) counts as an XFOIL failure. This is only a safety net: when the trailing-edge region matters (a flap, a thin section), constrain its thickness explicitly with `thickness(xs=[...])`.

- **Non-physical solutions.** XFOIL can converge, from a cold start as well as from a fallback, to a spurious solution whose drag is lower than its skin-friction drag ($C_D < C_{Df}$, negative pressure drag), with transition at the leading edge and reversed flow. The case met so far was on a crossed section, and over three optimizations (441 analyses) with the crossing test active, none was found on a valid section. The test costs no computation and is kept as a second safety net: a viscous result with $C_D < C_{Df}$ or a non-finite $C_D$ counts as an XFOIL failure.

- **Residual failures.** Two XFOIL stalls near transition (oscillation of the transition location, Newton frozen at the transition station) are handled by $\text{XFOIL}_{\text{AD}}$ without changing the equations. Isolated failures remain, mostly near stall; the original XFOIL fails on the same shapes.

- **Determinism.** A converged analysis does not depend on the history, but the *convergence* of a borderline shape can depend on the starting angle (CL mode). A design already analyzed without gradient is therefore re-evaluated with gradient starting from its converged angles.

- **Initial shape.** It must converge at all the points, including the points outside the objective. For a section that does not have to be symmetric, start from a cambered airfoil for the high-$C_L$ points, or from a non-zero flap deflection.

- $C_f \geq 0$ **and laminar bubbles.** XFOIL's $C_f$ is negative in laminar separation bubbles, which reattach. A `cfmin` constraint over the whole section forbids them and can make the problem very constrained, or even infeasible; restrict `cfmin_xrange` or `cfmin_side` to the separation you want to avoid (Section 10.4). The position of the bubble and of the transition varies irregularly with the shape: the constraint is piecewise differentiable, like the objective.

- **Flap.** Keep the hinge in a sufficiently thick region (`thickness` constraint at $x_h$ if needed). If `flap deflection ... folds the surface` appears, increase `flap_blend` or reduce `flap_bounds`. The faired flap is not XFOIL's FLAP command: at equal deflection, the results differ slightly near the hinge (small $C_p$ bumps over $x_h \pm b$).

- **Regime changes.** A trial shape can make the flow switch (bubble, separation, transition jump); spikes then appear in the history, and SLSQP backs off. They are limited by reducing `dv_bounds`, `dv_scale` or `flap_scale`.

- $e^N$ **transition.** The transition location depends irregularly on the shape: the function is piecewise differentiable and has local optima. Try several starting points, or force the transition (`xtr`) for a first pass. When the transition location oscillates from one iteration to the next, $\text{XFOIL}_{\text{AD}}$ stabilizes the Newton method; convergence is then slower (more iterations).

- **Ncrit, Re, Mach** are not design variables. The derivatives with respect to Re and $N_{crit}$ are available (`adjoint(..., flow=True)`); Re follows the chord with a chord model (§7.3); for an uncertainty, use robust optimization.

- **Cm.** Without a constraint, the optimizer often loads the rear of the airfoil and makes $C_M$ more negative; the flap accentuates this effect.
- **Checking the gradients** of a new problem by finite differences (example c04): expected errors of the order of $10^{-4}$ on the nodes and $10^{-6}$ or less on the design variables.

# 17 Related tools and licence

| Tool | Approach |
|---|---|
| CMPLXFOIL (MDO Lab) | XFOIL with complex step: exact, but cost $\propto$ number of variables |
| Xfoil.jl (BYU) | XFOIL in Julia, complex-step version |
| mfoil (Fidkowski) | rewrite of XFOIL in Matlab/Python, coupled Newton |
| NeuralFoil / AeroSandbox | differentiable neural surrogate, a few % from XFOIL |
| XOPTFOIL | XFOIL and gradient-free methods (PSO, genetic), flap through the FLAP command |
| SU2, DAFoam, elsA | CFD (RANS) adjoints |

The distinctive feature of $\text{XFOIL}_{\text{AD}}$ is the exact adjoint gradient of the whole of XFOIL, boundary layer included: its cost does not depend on the number of variables, flap deflections included.

$\text{XFOIL}_{\text{AD}}$ builds on XFOIL (M. Drela, H. Youngren), distributed under the GPL licence, and on its Fortran 90 port (D. de Vries). IPOPT (coin-or, Eclipse Public License 2.0) is not distributed with $\text{XFOIL}_{\text{AD}}$; it is only loaded if `method="ipopt"` is requested.